# EngageNet: Multi-Modal Engagement Regression with Ordered State-Space Fusion

Lado Turmanidze

October 23, 2026

**Abstract**

EngageNet is a multimodal state-space architecture for real-time engagement regression in dyadic interactions. It combines per-modality bidirectional selective state-space models (BiMamba) with a differentiable Gumbel-Sinkhorn permutation layer for inter-modal fusion, and outputs calibrated predictions via Beta regression heads. A test-time adaptation mechanism leveraging predictive variance as an entropy proxy enables (at least it should, in principle) robust deployment under distribution shift across corpora (NoXi, NoXi+J). The system is evaluated using the Concordance Correlation Coefficient (CCC) as the primary metric and the Conditional Demographic Disparity (CDD) as a fairness constraint.

## 1 Architecture

### 1.1 Input

Let $x = \{x_1, \ldots, x_M\}$ with $x_i \in \mathbb{R}^{C_i \times L}$ for all $i \in \{1, \ldots, M\}$, where $C_i$ is the number of acquisition channels for the $i$-th modality and $L$ represents the number of time steps of the input signal. The feature streams are delivered at different sampling rates (the W2v-BERT 2.0 embeddings of NoXi, for example, arrive at 40 Hz, most other streams at 25 Hz), so every stream is first resampled to a common 25 Hz by linear interpolation and then cut into windows of $L$ frames, consecutive windows overlapping by half. All $M$ inputs therefore share the same $L$; we use $L = 250$ frames, i.e. 10 s.

#### 1.1.1 Dyadic Structure and Role Streams

Each session in the NoXi [17] family of corpora - including the NoXi+J extension [18] - is a screen-mediated dyadic interaction between an *expert* and a *novice*, recorded independently (separate cameras, separate microphones). Every feature type therefore arrives as two parallel streams, one per role, and both carry their own frame-wise engagement annotation.

We denote by $\mathcal{F}$ the set of active feature types (e.g. eGeMAPS v2, W2v-BERT 2.0, OpenFace2, OpenPose) and by $\mathcal{R} = \{\text{expert}, \text{novice}\}$ the set of roles. The index $i$ in $x_i$ ranges over the Cartesian product $\mathcal{F} \times \mathcal{R}$, so that

$$M = |\mathcal{F}| \times |\mathcal{R}| = 2|\mathcal{F}|.$$

With the four core feature types this yields $M = 8$ input streams. The challenge release provides ten feature types; we tested adding the remaining six in groups (Section 4.5), and only the XLM-RoBERTa embedding of the transcript improved the results. The final configuration therefore uses $|\mathcal{F}| = 5$ feature types and $M = 10$ streams. Every subsequent occurrence of $M$ in this document - including the modality-ordering permutation of Section 2 - refers to these role-feature pairs, not to feature types alone.

**Weight sharing across roles.** The two roles observe the same physical signal semantics (an eGeMAPS vector means the same thing whether it was extracted from the expert's or the novice's microphone). Accordingly, the InitEncoder, the uniform channel projection, the intra-modal BiMamba block, and the unimodal Beta head are instantiated *once per feature type* and applied to both roles. Only the inter-modal stage distinguishes the roles, since the permutation is learned over all $M$ role-feature pairs jointly. This halves the parameter count relative to per-role encoders and forces the shared representation to be role-agnostic.

**Target construction.** Write $y_t^e$ and $y_t^n$ for the engagement annotations of the expert and the novice at frame $t$, each in $[0, 1]$. The challenge asks for both separately [13], and we predict both. Suppose both annotations have the same variance $\sigma^2$ and correlation $r$ with each other. Scaling a random variable by $\frac{1}{2}$ scales its variance by $\frac{1}{4}$, and the variance of a sum is the sum of the variances plus twice the covariance, so

$$\mathrm{Var}(\bar{y}) = \tfrac{1}{4}\left[\mathrm{Var}(y^e) + \mathrm{Var}(y^n) + 2\,\mathrm{Cov}(y^e, y^n)\right] = \tfrac{1}{4}\left[\sigma^2 + \sigma^2 + 2r\sigma^2\right] = \frac{\sigma^2(1+r)}{2}$$

using $\mathrm{Cov}(y^e, y^n) = r\sigma^2$. The corpus paper [18] measures exactly this correlation and finds it to be small and, in three of four language sets, negative: $r = -0.19$ for German, $-0.07$ for English, $-0.05$ for French, and $+0.51$ for Japanese, the last attributed to a stronger tendency toward interpersonal harmony. Substituting these values, the averaged target retains only 41%, 47% and 48% of the variance in the European sets respectively, and 76% in the Japanese set - so on most of the corpus more than half the signal is destroyed before training begins. Since CCC is normalised by the variance of the target (Section 3.1), what is flattened away here cannot be recovered later.

Predicting both roles needs no new machinery, because the two participants are never mixed on the way in: their recordings enter as separate streams and stay separately indexed throughout. The network therefore ends with one output head per role rather than one shared head, each producing that participant's engagement curve, and each auxiliary per-modality head is trained against the participant whose stream it reads.

### 1.1.2 Input Standardisation

The raw feature streams occupy vastly different numerical ranges: eGeMAPS v2 $\in [-2.6 \times 10^5, 2.6 \times 10^5]$ and OpenFace2 $\in [-3 \times 10^4, 3 \times 10^4]$, while W2v-BERT 2.0 embeddings lie within $[-4, 4]$. These magnitudes compound through the encoder and state-space stages, reaching $\sim 10^{11}$ at the cross-modal block and driving the Beta shape parameters to $\sim 10^{14}$, at which point the negative log-likelihood is numerically meaningless and optimisation fails.

We therefore standardise every channel to zero mean and unit variance before it enters the network:

$$\tilde{x}_i^{(c)} = \frac{x_i^{(c)} - \mu_i^{(c)}}{\sigma_i^{(c)}}, \qquad c \in \{1, \ldots, C_i\},$$

where the per-channel statistics are accumulated in double precision over the **training split only** and reused unchanged for validation and test. Channels whose standard deviation falls below $10^{-8}$ (constant channels) are centred but not rescaled, avoiding division by zero. Because the two roles share encoder weights, they necessarily share normalisation statistics as well. The statistics are computed once per corpus.

## 1.2 Output

For each frame $t$ of the encoded sequence the network emits two strictly positive shape parameters $(\alpha_t, \beta_t)$ that define a Beta predictive density over the bounded engagement scale,

$$p(y_t \mid x) = \mathrm{Beta}\big(\alpha_t(x),\, \beta_t(x)\big), \qquad y_t \in [0, 1].$$

The scalar point prediction is the predictive mean:

$$\hat{y}_t = \hat{\mu}_t = \frac{\alpha_t}{\alpha_t + \beta_t} \in (0, 1),$$

while the predictive variance supplies the uncertainty signal consumed by the test-time adaptation filter (Section 1.6). In contrast to [1], where $y$ is a discrete class label represented as a one-hot vector $p(y|x) \in \mathbb{R}^N$ over $N$ categories, engagement estimation is a *continuous bounded regression* problem: there is no category axis, and the network output for a window of length $L'$ has shape $(L', 2)$ rather than $(L', N)$.

**Multimodal and unimodal heads.** The network has two kinds of output heads. Both have the same internal structure - the two-layer perceptron of Section 1.5 - and both emit a Beta distribution per frame. They differ in what they read, and therefore in what their prediction can depend on.

The *multimodal* heads produce EngageNet's predictions. There is one per role, so a single forward pass yields $\big(\alpha_t^r, \beta_t^r\big)$ for each $r \in \mathcal{R}$. Both read the fused representation $H \in \mathbb{R}^{L' \times MC'}$ leaving the inter-modal module (Section 1.3.2), in which all $M$ streams of both participants have already exchanged information; the two heads differ only in their read-out weights. The expert's predicted engagement at frame $t$ can therefore draw on the expert's voice and face as well as on the novice's.

The *unimodal* heads sit one stage earlier, before any fusion. Each is attached to a single intra-modal representation $u_i \in \mathbb{R}^{L' \times C'}$, the output of the intra-modal module for stream $i$ alone (Section 1.3.1), and predicts $(\alpha_i, \beta_i)$ from that one stream - for example, the novice's engagement from the novice's OpenFace2 features only. A unimodal head therefore does not combine modalities at all: it answers the question *how well can this stream predict engagement on its own*, whereas a multimodal head answers it for all streams together. As with the encoders, one unimodal head is instantiated per feature type and shared by the two roles, so $|\mathcal{F}|$ heads give $M$ unimodal predictions per frame. These predictions are never submitted. They serve two purposes: deep supervision during training, which keeps every branch individually predictive (Section 1.8), and the teacher-student consistency objective of test-time adaptation, in which the multimodal head acts as the teacher of the unimodal ones (Sections 1.6 and 1.9).

## 1.3 Inner Workings

We need to capture contextual information from multimodal behavioural time series simultaneously, and a unidirectional approach cannot fully capture the complex temporal patterns. The multimodal BiMamba network consists of two modules: the intra-modal BiMamba module and the inter-modal BiMamba module. The former is designed to model each modality independently, while the latter effectively fuses multimodal features.

### 1.3.1 Intra-modal BiMamba Module

First, an initial encoder $h_i = \mathrm{InitEncoder}_i(\tilde{x}_i) \in \mathbb{R}^{L' \times C'}$ for all $i \in \{1, \ldots, M\}$ is used to extract shallow features from a specific modality, where $C'$ is the number of output channels and $L'$ is the feature length of the output signal. Both are the same for every stream, and both are set by us. $C' = 128$ is imposed by the uniform channel projection $f_i$ at the end of each encoder

(below), which maps every stream to the same width regardless of its raw dimension $C_i$ (from 88 channels for eGeMAPS v2 to 1024 for W2v-BERT 2.0). $L'$ follows from the window length and the encoder stride, $L' = L/s_i$; we use stride $s_i = 1$ for every stream, so no temporal downsampling takes place and $L' = L = 250$ frames (10 s at 25 Hz) for all of them.

The window length is a design choice rather than a derived quantity. Ten seconds is long compared with the time engagement takes to change, which is a matter of seconds, and short compared with a session, which therefore yields more than a hundred training windows on average. Doubling it to 20 s ($L = 500$, window hop 250) made no measurable difference: over three seeds each, the NoXi mean moved from 0.709 to 0.714, and the NoXi+J mean from 0.529 to 0.560 with a seed-to-seed standard deviation of 0.054 (Section 4). We therefore kept the cheaper 10 s window.

The transposition of dimensions from $C_i \times L$ to $L' \times C'$ physically aligns the data structure with the sequence-first requirement of autoregressive models. By enforcing the sequence length $L'$ as the primary operational dimension, the subsequent state-space and 1D convolutional operators can process the temporal dimension across the channel feature space.

**InitEncoder internals.** Each InitEncoder is a shallow three-stage block applied to the standardised input:

$$h_i = \text{InitEncoder}_i(\tilde{x}_i) = f_i\Big(\sigma\big(\text{BN}\big(\text{Conv1D}_i(\tilde{x}_i^\top)\big)\big)\Big),$$

where $\text{Conv1D}_i$ is an ordinary (dense) temporal convolution mapping $C_i \to C_i'$ channels: every output channel at frame $t$ is a learned weighted sum over *all* input channels and a short window of neighbouring frames,

$$\big(\text{Conv1D}_i(z)\big)_{t,c'} = b_{c'} + \sum_{j=0}^{k_i-1} \sum_{c=1}^{C_i} W_{j,c,c'}\; z_{t+j-p_i,\,c}, \qquad p_i = \Big\lfloor \frac{k_i - 1}{2} \Big\rfloor, \qquad c' = 1, \ldots, C_i',$$

where $z = \tilde{x}_i^\top \in \mathbb{R}^{L \times C_i}$, frames outside the window count as zero (`SAME` padding), and $W \in \mathbb{R}^{k_i \times C_i \times C_i'}$ and $b \in \mathbb{R}^{C_i'}$ are learned. We use the same kernel for every stream: $k_i = 5$ frames (200 ms), so $p_i = 2$ and the filter is centred on frame $t$, covering frames $t-2, \ldots, t+2$. The stride is $s_i = 1$, and $C_i' = 64$ for eGeMAPS v2 and OpenPose and 128 for the other streams. This layer is the only place where a stream's raw channels are mixed with each other, and with $k_i C_i C_i'$ weights it is also the largest layer of the model for the wide streams: the convolutions of W2v-BERT 2.0, OpenFace2 and XLM-RoBERTa alone hold about 1.6M of the final model's 2.75M parameters. Further, BN is batch normalisation [16] over the channel axis, whose running statistics are frozen at evaluation time and which later serves as a surgical adaptation target (Section 1.10); $\sigma(x) = \frac{x}{1+e^{-x}}$ is the SiLU activation function, and $f_i$ is the uniform channel projection defined in Section 2.2, mapping $C_i' \to C'$ so that all modalities share a common width. Encoder weights are shared between the expert and novice streams of the same feature type.

The BiMamba module operates on $h_i$ as follows. Let $\text{LN}(\cdot)$ denote layer normalisation [15].

1. **Pre-normalisation.** The block input is normalised before any projection:

$$\bar{h}_i = \text{LN}(h_i) \in \mathbb{R}^{L' \times C'}.$$

   Here *depth* is the position of a block in the stack: the intra-modal block sits at depth one and the inter-modal (cross-modal) block at depth two, each fed by the output of the one before. Because every block adds its own output to its input (the residual connection of step 4), the signal handed from block to block - the *residual stream* - is a running sum that nothing rescales. Without this step each block would receive its input at whatever

magnitude the blocks before it happened to produce, and since each block can enlarge that magnitude further, magnitudes compound multiplicatively along the stack. Empirically the absolute activation maximum grew from $\sim 10$ at the frontend to $\sim 6 \times 10^{11}$ at the cross-modal stage, saturating the Beta heads and destroying the gradient signal. Pre-normalisation keeps the block input at unit scale while leaving the residual path (step 4) untouched.

2. The output of the gating mechanism is $g_i = \sigma(W_i^g \bar{h}_i + b_i^g)$, where $W_i^g$ is the weight matrix, $b_i^g$ is the bias, and $\sigma(x) = \dfrac{x}{1 + e^{-x}}$. Intuitively, $g_i$ highlights engagement-relevant information while suppressing noise and redundancy.

3. BiMamba models temporal dependencies from both forward and backward perspectives using state-space modeling, thereby capturing richer contextual dynamics:

$$h_i^{\rightarrow} = g_i \odot \mathrm{SSM}_{\rightarrow}(\sigma(\mathrm{DWConv1D}_{\rightarrow}(W_i^h \bar{h}_i + b_i^h))) \in \mathbb{R}^{L' \times C'}$$

$$h_i^{\leftarrow} = g_i \odot \mathrm{Rev}_t\Big(\mathrm{SSM}_{\leftarrow}(\sigma(\mathrm{DWConv1D}_{\leftarrow}(\mathrm{Rev}_t(W_i^h \bar{h}_i + b_i^h))))\Big) \in \mathbb{R}^{L' \times C'}$$

where $h_i^{\rightarrow}$ and $h_i^{\leftarrow}$ are the feature representations of the $i$-th modality in forward and backward modeling. $\mathrm{Rev}_t(\cdot)$ denotes flipping along the time dimension $L'$.

Note that the time reversal is applied to the backward SSM output *before* gating, so that the gate $g_i$ - which is computed once, in forward time order - modulates the feature belonging to the same time step $t$ in both branches. Applying $\mathrm{Rev}_t$ after gating would instead pair the gate at step $t$ with the feature at step $L' - t$, misaligning the two.

The operator $\odot$ denotes the Hadamard product, the element-wise multiplication of two arrays of the same shape, so the dimensions remain unchanged. $W_i^h$ represents the weight matrix and $b_i^h$ represents the bias.

DWConv1D is a *depthwise* temporal convolution: unlike the dense convolution of the InitEncoder, every channel is filtered separately with its own short kernel, and channels are not mixed. For an input $z \in \mathbb{R}^{L' \times C'}$,

$$\big(\mathrm{DWConv1D}(z)\big)_{t,d} = b_d + \sum_{j=0}^{D_C - 1} w_{j,d}\, z_{t+j-p,\,d}, \qquad p = \Big\lfloor \frac{D_C - 1}{2} \Big\rfloor, \qquad d = 1, \ldots, C',$$

with learned $w \in \mathbb{R}^{D_C \times C'}$ and $b \in \mathbb{R}^{C'}$, and frames outside the window counted as zero. Each output value therefore depends only on the same channel at $D_C$ neighbouring frames: it is a learned local smoothing or differencing of every channel, applied before the state-space model. We use $D_C = 4$, as in [2], so with $p = 1$ the filter covers frames $t-1, \ldots, t+2$; the backward branch runs on the reversed sequence, where the same filter covers frames $t - 2, \ldots, t + 1$ of the original time order. The reference Mamba implementation pads causally, so that frame $t$ sees only the past; since our model is bidirectional and processes complete windows, we use centred `SAME` padding instead. The two branches have separate convolutions. A depthwise filter has $D_C C'$ weights instead of the $D_C C'^2$ of a dense convolution, which keeps its cost linear in the channel width. In the inter-modal block, whose sequence runs over modalities instead of time, the same operator is used with $D_C = 2$, so each position is combined with the next one.

SSM$(\cdot)$ represents the Selective State Space Model. Its starting point is a continuous linear time-invariant (LTI) system, defined by the state equation $\xi'(t) = A\xi(t) + Bz(t)$ and the output equation $y(t) = C\xi(t)$, where $z(t)$ is the input signal. (Throughout, we write $z$ for the input of the SSM, to keep it apart from the block output $u_i$.) To operate

on discrete sequence steps, the continuous transition operator $A$ and input operator $B$ are discretised with a timestep $\Delta > 0$. The exact discretisation yields $\bar{A} = \exp(\Delta A)$ and $\bar{B} = (\Delta A)^{-1}(\bar{A} - I)\Delta B$. Following the reference Mamba implementation [2], we adopt the first-order (Euler) simplification of the input operator,

$$\bar{A} = \exp(\Delta A), \qquad \bar{B} \approx \Delta B,$$

which is accurate to $O(\Delta^2)$ and avoids the matrix inverse $(\Delta A)^{-1}$. Here $\exp(\cdot)$ is the matrix exponential, $\exp(X) = \sum \frac{X^k}{k!}$. Because $A$ is diagonal (see below), every power of $\Delta A$ is again diagonal, with the diagonal entries raised to that power, so the series acts on each diagonal entry separately:

$$\exp\big(\Delta \operatorname{diag}(a_1, \ldots, a_N)\big) = \operatorname{diag}\big(e^{\Delta a_1}, \ldots, e^{\Delta a_N}\big).$$

**The SSM, exactly.** Collecting the pieces described in the following paragraphs - the diagonal state matrix, its parameterisation, and the input-dependent $\Delta_t$, $B_t$, $C_t$ of the selection mechanism - the operator applied in each branch is the following. Its input is a sequence $z = (z_1, \ldots, z_{L'})$ with $z_t \in \mathbb{R}^{C'}$; in step 3, $z = \sigma(\mathrm{DWConv1D}(W_i^h \bar{h}_i + b_i^h))$. Every channel $d \in \{1, \ldots, C'\}$ carries its own state $\xi_t^{(d)} \in \mathbb{R}^N$, started at $\xi_0^{(d)} = \mathbf{0}$, and

$$\xi_t^{(d)} = \exp\big(\Delta_{t,d}\, a_d\big) \odot \xi_{t-1}^{(d)} + \Delta_{t,d}\, z_{t,d}\, B_t, \qquad t = 1, \ldots, L',$$

$$\big(\mathrm{SSM}(z)\big)_{t,d} = y_{t,d} = C_t^\top \xi_t^{(d)} = \sum_{n=1}^{N} C_{t,n}\, \xi_{t,n}^{(d)}.$$

Here $a_d = (a_{d,1}, \ldots, a_{d,N}) \in \mathbb{R}^N$ holds the (negative) decay rates of channel $d$ and the exponential is taken elementwise; $\Delta_{t,d} > 0$ is the timestep of channel $d$ at step $t$; and $B_t, C_t \in \mathbb{R}^N$ are the input and read-out vectors at step $t$, shared by all channels. The first line is the discretised state equation, with $\bar{A} = \exp(\Delta A)$ and $\bar{B} = \Delta B$; the second is the output equation. The SSM thus maps $\mathbb{R}^{L' \times C'}$ to $\mathbb{R}^{L' \times C'}$.

**No feedthrough term.** The general state-space model has a second term in its output equation, $y(t) = C\xi(t) + Dz(t)$. This *feedthrough* is a direct path from the current input to the current output that bypasses the state. The Mamba paper writes the output equation without it, but its reference implementation does add it, as a learned vector with one entry per channel [2]; BiMamba-TTA [1] does not write out the SSM at all, so its paper gives no indication either way. Our implementation omits it, i.e. $D = 0$. This was not a deliberate design decision but a consequence of implementing the equations as written. A path around the state still exists at block level, because the residual connection of step 4 adds the block input $h_i$ to the block output; what is missing is only the narrower skip from the SSM's own input $z$ to its output. We have not tested whether adding it changes the results.

**Diagonal state transition.** Following [2], the state-space model is applied independently to each of the $C'$ channels, and the state matrix of every channel is restricted to be *diagonal*. Channel $d$ thus carries $N$ state coordinates evolving under $A^{(d)} = \operatorname{diag}(a_{d,1}, \ldots, a_{d,N})$, and because a diagonal matrix is fully determined by its diagonal, no $N \times N$ array is ever materialised: the entire parameter collapses to the $C' \times N$ array $A = (a_{d,n})$, one row of $N$ entries per channel. Each entry governs how quickly one state coordinate forgets, which is why we refer to these entries as decay rates. The update is then elementwise rather than a matrix product, costing $O(N)$ per channel per step instead of $O(N^2)$ and storing $C'N$ parameters instead of $C'N^2$. Diagonality is not merely an efficiency measure: it is what allows the recurrence to be evaluated by a parallel scan, as shown below.

**Parameterisation of $A$.** The state transition matrix is not learned directly. Instead we learn $\log(-A)$ and recover

$$A = -\exp\bigl(A_{\log}\bigr) \in \mathbb{R}^{C' \times N} \quad \text{(elementwise)},$$

with $A_{\log}$ initialised by the S4D-Real scheme described below. This parameterisation constrains $A < 0$ (elementwise). This guarantees $\bar{A} = \exp(\Delta A) \in (0, 1)$, so the recurrence is contractive and the hidden state cannot diverge - a stability property that an unconstrained $A$ does not provide. Separate $A_{\log}$ parameters are maintained for the forward and backward directions.

**Temporal receptive field.** The decay rate and the timestep jointly determine how current input can influence future output. Consider a single state coordinate, which follows the recurrence

$$\xi_t = \bar{A}\,\xi_{t-1} + b_t, \qquad b_t := \bar{B}_t z_t,$$

which is *first-order*: the state at step $t$ is determined solely by the previous state $\xi_{t-1}$ and the current input alone, with no explicit dependence on $\xi_{t-2}$ or earlier. Suppose the input does not exist after step $t_0$ ($z_t = 0 \implies b_t = 0$) for $t > t_0$. Unrolling from $\xi_{t_0}$,

$$\xi_{t_0+1} = \bar{A}\,\xi_{t_0}, \qquad \xi_{t_0+2} = \bar{A}^2\xi_{t_0}, \qquad \ldots, \qquad \xi_{t_0+k} = \bar{A}^{\,k}\,\xi_{t_0}.$$

Since $\bar{A} = \exp(\Delta A) \in (0, 1)$, the retained state shrinks geometrically; $\bar{A}$ is called the *decay* (or *retention*) factor. Equivalently, and this is the reading we need, the influence of an input entering at step $t_0$ on the state $k$ steps later carries the weight $\bar{A}^{\,k}$: this is exactly the accumulated multiplier that appears in the associative scan derivation below. The *memory* of the block is described by how quickly these weights $\bar{A}^{\,1}, \bar{A}^{\,2}, \bar{A}^{\,3}, \ldots$ shrink as $k$ grows, and its *effective memory* $\tau_{\text{eff}}$ is the conventional single-number summary - the number of steps after which the weight has fallen to $e^{-1}$ of its initial value. This $e^{-1}$ convention is the standard definition of the time constant of an exponentially decaying response in linear systems theory [20], and we adopt it unchanged. Solving

$$\bar{A}^{\,\tau_{\text{eff}}} = e^{-1} \implies \tau_{\text{eff}} \ln \bar{A} = -1 \implies \tau_{\text{eff}} = \frac{-1}{\ln \bar{A}},$$

and substituting $\ln \bar{A} = \Delta A = -\Delta |A|$ (recall $A < 0$ by construction),

$$\boxed{\tau_{\text{eff}} = \frac{1}{\Delta\,|A|}}$$

measured in units of encoder output steps.

**Why the initialisation matters.** The naive choice - and the initialisation of our first implementation - is $A_{\log} = \mathbf{0}$ and $s_\Delta = \mathbf{0}$. It gives $|A| = \exp(0) = 1$ and, since the input-dependent projection contributes negligibly at initialisation, $\Delta \approx \text{softplus}(0) = \ln 2 \approx 0.693$. Hence

$$\tau_{\text{eff}} = \frac{1}{0.693 \times 1} \approx 1.44 \text{ steps}$$

The output sequence retains the input rate of 25 Hz and one step spans $1/25$ s $= 40$ ms; the receptive field is therefore

$$1.44 \times 40\,\text{ms} \approx 58\,\text{ms}$$

A uniform $A_{\log}$ compounds the difficulty: every one of the $N$ state coordinates then carries the same decay rate, so the state dimension supplies $N$ copies of a single timescale rather than a spread of them, entirely defeating the purpose of $N > 1$.

**S4D-Real initialisation.** We therefore adopt the real-valued diagonal initialisation of [19], used in the reference Mamba implementation [2]: setting

$$A_{\log}[d,n] = \ln n \quad \rightarrow \quad a_{d,n} = -n, \qquad n = 1, \ldots, N,$$

gives every channel the decay rates $1, 2, \ldots, N$ ($N = 16$) - a ladder of distinct timescales rather than a single one. The timestep bias is initialised so that $\Delta$ is log-uniform on $[\Delta_{\min}, \Delta_{\max}] = [10^{-3}, 10^{-1}]$, obtained by inverting the softplus,

$$s_\Delta = \ln\left(e^{\Delta} - 1\right),$$

which is the exact inverse since $\text{softplus}(s_\Delta) = \ln(1 + e^{s_\Delta}) = \Delta$. The resulting receptive fields $\tau_{\text{eff}} = \frac{1}{\Delta|A|}$ span from a fraction of a step to of order $10^3$ steps - tens of seconds at 25 Hz - allowing the selection mechanism to choose an integration horizon per channel and per input rather than beginning pinned to a single very short one.

Unlike traditional LTI systems, Mamba introduces *selection* by making the discretized parameters $(\bar{B}, C, \Delta)$ functions of the input $z_t$. Specifically:

$$\Delta_t = \text{softplus}(s_\Delta + \text{Linear}(z_t)) \in \mathbb{R}^{C'}$$

$$B_t = \text{Linear}(z_t) \in \mathbb{R}^N, \quad C_t = \text{Linear}(z_t) \in \mathbb{R}^N$$

where $\text{softplus}(x) = \ln(1 + e^x)$ and $s_\Delta \in \mathbb{R}^{C'}$ is a learned, input-independent bias term that defines the baseline timescale of the system. The inclusion of $s_\Delta$ within the *softplus* function ensures that the resulting $\Delta_t$ remains strictly positive, which is required for the stability of the discretization. This selection mechanism allows the discrete transition $\xi_t = \bar{A}_t \xi_{t-1} + \bar{B}_t z_t$ to dynamically adjust its temporal resolution. The projections producing $B_t$ and $C_t$ are bias-free.

**Parallel evaluation via associative scan.** This part is about speeding up the computation of the states $\xi_1, \ldots, \xi_{L'}$, which is the only expensive part of the SSM. Written for all channels at once, the states follow the recurrence introduced above,

$$\xi_t = \bar{A}_t \odot \xi_{t-1} + \bar{B}_t z_t, \qquad t = 1, \ldots, L', \qquad \xi_0 = \mathbf{0},$$

where $\xi_t$, $\bar{A}_t$ and $\bar{B}_t z_t$ are $C' \times N$ arrays with entries $\xi_{t,n}^{(d)}$, $\exp(\Delta_{t,d}\, a_{d,n})$ and $\Delta_{t,d}\, z_{t,d}\, B_{t,n}$. Once the states are known, the output $y_{t,d} = \sum_{n=1}^{N} C_{t,n}\, \xi_{t,n}^{(d)}$ is a cheap read-out at every step separately. $C_{t,n}$ is the $n$-th entry of the read-out vector $C_t = \text{Linear}(z_t) \in \mathbb{R}^N$ produced by the selection mechanism: the weight with which state coordinate $n$ contributes to the output at step $t$, the same for every channel. Evaluated naively the recurrence is a loop: $\xi_t$ cannot be formed until $\xi_{t-1}$ exists, so the $L'$ steps are strictly serial. The math for each step is very simple (just a multiply-add operation on a $C' \times N$ array), so calculating it one step at a time wastes GPU time on just starting the tasks and leaves the hardware mostly idle. For a sequence of $L' = 250$, this means waiting to sequentially launch 250 separate tasks for an amount of work the GPU could otherwise process almost instantly. Ultimately, the step-by-step dependency is what slows the process down, not the calculations themselves.

Two structural facts make a parallel evaluation possible. First, because $A$ is *diagonal* in the state dimension, the update does not mix state coordinates: the vector recurrence decomposes into $n_{\text{batch}} \times C' \times N$ mutually independent *scalar* recurrences, one per (sample, channel, state) triple, all sharing the same structure. Second, each such scalar recurrence is an *affine* map of the previous state. Write the step-$t$ update as the function

$$f_t : \mathbb{R} \to \mathbb{R}, \qquad f_t(\xi) = a_t\, \xi + b_t, \qquad a_t := \bar{A}_t, \quad b_t := \bar{B}_t z_t,$$

so that unrolling from $\xi_0 = 0$ gives $\xi_t = (f_t \circ f_{t-1} \circ \cdots \circ f_1)(0)$. The state at time $t$ is therefore nothing more than the composition of the first $t$ update maps, evaluated at zero - and computing *every* prefix of a sequence under some combining operation is precisely what a *scan* (or prefix computation) does. The archetypal example is the prefix sum, which turns $(x_1, \ldots, x_n)$ into $(x_1,\ x_1 + x_2,\ \ldots)$; a scan generalises this to any binary operator $\oplus$.

The reason this buys anything is that a scan under an *associative* $\oplus$ need not be computed left to right. Associativity means partial results may be regrouped freely, so the sequence can be reduced pairwise in a balanced binary tree: total work stays $O(L')$, but the *sequential depth* - the length of the longest chain of dependent operations - falls from $O(L')$ to $O(\log L')$. For $L' = 250$ that is roughly 8 dependent stages instead of 250, each stage saturating the device with independent work.

*Closure and the composition rule.* To scan over the update maps we must be able to represent a composed map by a finite parameter set. Affine maps in one variable are closed under composition, and the family is parameterised by exactly two numbers - a multiplier and an offset - which is why the scan carries *pairs* rather than scalars. Explicitly, for $i < j$,

$$(f_j \circ f_i)(\xi) = f_j\big(f_i(\xi)\big) = a_j\big(a_i\xi + b_i\big) + b_j = \underbrace{(a_j a_i)}_{\text{new multiplier}}\ \xi + \underbrace{(a_j b_i + b_j)}_{\text{new offset}}.$$

The composite is again affine, so identifying each map with its parameter pair $(a, b)$ induces the binary operator

$$(a_i, b_i) \oplus (a_j, b_j) := (a_j a_i,\ a_j b_i + b_j), \qquad i < j,$$

where the left operand is the earlier segment of the sequence. Intuitively the two components accumulate different things: the multiplier is the running product of decay factors, recording how much of the initial state survives, while the offset is the running total of inputs, each discounted by all the decay factors applied after it entered.

*Associativity.* The operator inherits associativity from function composition, but it is worth verifying directly on the parameters. Expanding the two groupings,

$$\big[(a_i, b_i) \oplus (a_j, b_j)\big] \oplus (a_k, b_k) = (a_j a_i,\ a_j b_i + b_j) \oplus (a_k, b_k) = \big(a_k a_j a_i,\ a_k a_j b_i + a_k b_j + b_k\big),$$

$$(a_i, b_i) \oplus \big[(a_j, b_j) \oplus (a_k, b_k)\big] = (a_i, b_i) \oplus (a_k a_j,\ a_k b_j + b_k) = \big(a_k a_j a_i,\ a_k a_j b_i + a_k b_j + b_k\big),$$

which agree componentwise. Together with the identity element $(1, 0)$ - the identity map $\xi \mapsto \xi$ - the parameter pairs form a monoid under $\oplus$, which is exactly the algebraic structure a scan requires. Note that $\oplus$ is *not* commutative: $(a_j a_i, a_j b_i + b_j) \neq (a_i a_j, a_i b_j + b_i)$ in general, reflecting the obvious fact that reordering the timesteps changes the trajectory. This costs us nothing, since a scan requires associativity but never commutativity.

Running the scan over $\big((a_1, b_1), \ldots, (a_{L'}, b_{L'})\big)$ yields at position $t$ the pair representing $f_t \circ \cdots \circ f_1$; because $\xi_0 = 0$ the multiplier component annihilates and the state is read off directly from the offset component, $\xi_t = a_t \cdots a_1 \cdot 0 + (\text{offset}) = (\text{offset})$.

*Implementation.* JAX exposes this primitive as `jax.lax.associative_scan(fn, elems, axis)`, where `fn` implements $\oplus$ on a pair of PyTrees and is required (not merely assumed) to be associative. We supply the tuple $(\bar{A},\ \bar{B}z)$, both of shape $(L', n_{\text{batch}}, C', N)$ after moving the sequence axis to the front, and scan along `axis=0`; the operator is applied elementwise, so the $n_{\text{batch}} \times C' \times N$ independent recurrences are resolved simultaneously in a single call. The same routine serves the inter-modal block unchanged, where the

scanned axis is the modality sequence of length $M$ rather than time, with the $L'$ time steps folded into the batch axis.

Finally, the log-space parameterisation of $A$ interacts favourably with the scan. Since $\bar{A}_t = \exp(\Delta_t A) \in (0, 1)$, the cumulative multipliers $a_t \cdots a_1$ form a decreasing product and cannot overflow, no matter how long the sequence; the worst case is underflow to zero, which is the gentle behaviour of a state that has simply forgotten its distant past. An unconstrained $A$ permitting $|\bar{A}| > 1$ would instead make these partial products grow geometrically in the sequence length, and the tree-structured reduction would surface that divergence just as surely as a serial loop would.

4. BiMamba applies a linear projection and a residual connection to integrate bidirectional features and stabilize training:

$$u_i = h_i + \left( W_i^o \left( \frac{h_i^{\rightarrow} + h_i^{\leftarrow}}{2} \right) + b_i^o \right) \in \mathbb{R}^{L' \times C'}$$

which is the output of the intra-modal BiMamba module for the $i$-th modality. $W_i^o$ is the weight matrix, and $b_i^o$ is the bias. Observe that the residual carries the *unnormalised* $h_i$, not $\bar{h}_i$: the layer normalisation of step 1 conditions the block's internal computation without rescaling the identity path.

### 1.3.2 Inter-modal BiMamba Module

Relying solely on intra-modal modeling neglects inter-modal correlations that are obviously important for comprehensive multimodal understanding. The intra-modal stage yields $M$ representations $u_i \in \mathbb{R}^{L' \times C'}$ of common width, and the fusion stage must let them exchange information. How that exchange is arranged - specifically, which axis the state-space model treats as its sequence - determines both what the fusion can express and whether the learned modality ordering of Section 2 has any effect at all. We first record why two natural arrangements are unsatisfactory, then give the one we adopt.

**Arrangement (i): concatenate along channels, scan over time.** Form

$$U^{(t)} = u_1^{(t)} \circ u_2^{(t)} \circ \cdots \circ u_M^{(t)} \in \mathbb{R}^{MC'},$$

where $\circ$ denotes concatenation and $u_i^{(t)} \in \mathbb{R}^{C'}$ is the $i$-th modality's representation at time $t$, and apply the BiMamba block along the time axis with feature width $MC'$. Note that $\circ$ places the $M$ vectors end to end, producing one long vector of length $MC'$ in which entries $(i-1)C' + 1, \ldots, iC'$ belong to modality $i$. This is different from stacking them as the rows of an $M \times C'$ matrix, which is what arrangement (iii) below does. Modalities then interact only through the block's dense projections $W \in \mathbb{R}^{MC' \times MC'}$, which mix all channels of all modalities at every time step. The fusion is expressive but structurally unconstrained, and it has two problems.

First, any custom reordering of the input modalities becomes useless. Because the reordering operation happens right before a dense layer (a linear weight matrix), the network simply absorbs the fixed rearrangement directly into its weight calculations without changing the model's capabilities. The only benefit comes from making the reordering dynamic (input-dependent), but even then, it creates a soft blend of modalities rather than a true sequential order. This defeats the original goal of learning an ordering, which relies on processing the modalities step-by-step.

Formally, let $P \in \mathbb{R}^{M \times M}$ be the reordering matrix of Section 2.2, and let $I_{C'}$ be the $C' \times C'$ identity matrix - the subscript only states its size. Their Kronecker product $P \otimes I_{C'} \in \mathbb{R}^{MC' \times MC'}$

is the block matrix made of $M \times M$ blocks of size $C' \times C'$, whose block $(i, j)$ is $P_{ij} I_{C'}$. For $M = 2$ and the permutation that swaps the two modalities,

$$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P \otimes I_{C'} = \begin{pmatrix} \mathbf{0} & I_{C'} \\ I_{C'} & \mathbf{0} \end{pmatrix},$$

so multiplying $U^{(t)}$ by it exchanges the two halves of length $C'$: it moves whole modalities while leaving the order of the channels inside each modality unchanged. Reordering before the dense projection then gives $W(P \otimes I_{C'})U^{(t)} = W'U^{(t)}$ with $W' := W(P \otimes I_{C'})$, so for any fixed $P$ the reordered model computes exactly what the unreordered model computes with weights $W'$: the two are the same model class, and the permutation is unlearnable because it is already absorbed.

Second, the parameter count grows quadratically with the number of modalities ($M$). Combining all modalities into a single wide feature dimension causes the fusion layer to require roughly $M^2$ times as many parameters. This makes the fusion stage take up most of the network's capacity, which easily leads to severe overfitting on smaller datasets containing only a few dozen sessions.

**Arrangement (ii): concatenate along channels, scan over the concatenated axis.** A second approach treats the combined index as the sequence, scanning over all $MC'$ positions so each modality forms a block of $C'$ steps. This fails due to an information bottleneck, which becomes clear when we unroll the recurrence over the scan index $k$:

$$\xi_k = \bar{A}_k \, \xi_{k-1} + b_k, \qquad b_k := \bar{B}_k s_k, \qquad \xi_0 = \mathbf{0},$$

where $k$ is the position, $s_k$ is the input, $\xi_k$ is the hidden state, $\bar{A}_k$ is the decay factor, and $\bar{B}_k$ is the input operator. Unrolling this step-by-step gives:

$$\begin{aligned}
\xi_1 &= \bar{A}_1 \cdot \mathbf{0} + b_1 = b_1, \\
\xi_2 &= \bar{A}_2 \xi_1 + b_2 = \bar{A}_2 b_1 + b_2, \\
\xi_3 &= \bar{A}_3 \xi_2 + b_3 = \bar{A}_3 \bar{A}_2 b_1 + \bar{A}_3 b_2 + b_3, \\
&\vdots \\
\xi_k &= \sum_{k'=1}^{k} \left( \prod_{l=k'+1}^{k} \bar{A}_l \right) b_{k'},
\end{aligned}$$

with an empty product equaling 1. The product $\prod_{l=k'+1}^{k} \bar{A}_l$ is the weight with which the input $b_{k'}$ still contributes to the state at position $k$. Each factor $\bar{A}_l = \exp(\Delta_l A) \in (0, 1)$ is a *retention factor*: the fraction of the state that survives one step, from position $l-1$ to position $l$ ($\bar{A}_l = 0.9$ keeps 90% of the state and forgets 10%). An input travelling from position $k'$ to position $k$ is multiplied by one such factor for every step in between, so the share of it that arrives is their product; with a constant $\bar{A} = 0.9$, only $0.9^{10} \approx 0.35$ of it is left after 10 steps. At the start of training, the model's ability to change this factor depending on the input is weak, so the retention factor stays close to a constant baseline, $\bar{A}$. Because it is roughly constant, multiplying it for every step in the gap simplifies to $\bar{A}$ raised to the power of the distance between positions:

$$\prod_{l=k'+1}^{k} \bar{A}_l \approx \bar{A}^{\,k-k'}$$

Under this arrangement, one modality is separated from the next by $C'$ steps, so the information passed between them shrinks by a factor of $\bar{A}^{\,C'}$. Since $\bar{A}$ is a fraction, raising it to a large power like $C'$ shrinks the surviving information to practically zero. To successfully pass information

across this gap, the network would have to push $\bar{A}$ incredibly close to 1. However, doing so stops the memory from fading, causing the block to lose its ability to distinguish different positions along the sequence. Furthermore, the gradient signal needed to increase $\bar{A}$ is blocked by this exact same bottleneck.

**Arrangement (iii), adopted: scan over the modality axis.** Both of the previous problems are solved if we treat the list of modalities itself as the sequence. Instead of a single modality taking up $C'$ steps, it now takes up exactly one position. First, the modality outputs are reordered by the learned matrix $P$ of Section 2.2: position $i$ of the sequence receives $\tilde{u}_i = \sum_{j=1}^{M} P_{ij} u_j \in \mathbb{R}^{L' \times C'}$. At every individual time step $t$, the model processes a sequence of $M$ vectors, $\big(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)}\big)$, each with a size of $C'$. The BiMamba block is then applied directly across this sequence of modalities:

$$H^{(t)} = \mathrm{BiMamba}_{\mathrm{modality}}\big(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)}\big) \in \mathbb{R}^{M \times C'}, \qquad t = 1, \ldots, L'.$$

This is calculated independently for each time step using the same shared weights. The job of understanding time is left entirely to a previous part of the network.

Because modalities are now sitting directly next to each other in the sequence, the memory decay between them is only $\bar{A}$ instead of the massive $\bar{A}^{C'}$ bottleneck. This allows information to flow easily between them, no matter what decay rate the model learns. When scanning forward, modality $i$ gathers context from the modalities placed before it, $\{1, \ldots, i-1\}$. When scanning backward, it gathers context from the modalities placed after it, $\{M, \ldots, i+1\}$. Averaging these two directions guarantees that every modality can "see" all the others.

This setup fixes the core issues in two major ways:

- **The sorting mechanism works:** Because the fusion process is now sensitive to order, the learned permutation actively decides which modalities give context to which. This effect can no longer be bypassed or absorbed by standard network weights.
- **Efficiency gains:** The feature width shrinks to $C'$, which drops the parameter cost from a heavy $\Theta(M^2 C'^2)$ down to just $\Theta(C'^2)$. Additionally, the depth of the dependent math operations (the associative scan) drops from $\lceil \log_2 L' \rceil$ to just $\lceil \log_2 M \rceil$, running on a much smaller state.

Before the heads, every slot is returned to the stream it came from. Slot $i$ of the scan holds $\sum_j P_{ij} u_j$, so stream $j$ receives back $\hat{H}_j^{(t)} = \sum_{i=1}^{M} P_{ij} H_i^{(t)}$, i.e. the output is multiplied by $P^\top$, which for a permutation is its inverse. Without this step the layout of the heads' input would change from window to window: under an input-dependent $P$, the part of $H$ that the read-out weights associate with slot 3 would hold, say, the expert's face in one window and the novice's text in the next. With it, part $j$ of $H$ always belongs to stream $j$, and the order influences only the scan, where it is meant to. Finally, the output is stitched back together side-by-side to match the expected format for the final prediction heads:

$$H^{(t)} = \hat{H}_1^{(t)} \circ \cdots \circ \hat{H}_M^{(t)} \in \mathbb{R}^{MC'}$$

which returns the data to its original shape of $H \in \mathbb{R}^{L' \times MC'}$.

### 1.4 Differential Entropy and the Failure of Gaussian Assumptions

To adapt the entropy-based selection mechanism of BiMamba-TTA for continuous engagement regression, we transition from Shannon entropy to **differential entropy**. For a continuous random variable $y$ with probability density $p(y)$, the differential entropy is defined as:

$$H(y) = -\int_{\mathcal{Y}} p(y) \ln p(y)\, dy$$

While a Gaussian assumption $p(y|x) = \mathcal{N}(\mu, \sigma^2)$ simplifies this to $H(y) = \frac{1}{2}\ln(2\pi e\sigma^2)$, it is fundamentally inappropriate for engagement modeling. Engagement scores are strictly bounded within the interval $[0, 1]$. A Gaussian distribution assigns non-zero probability mass to values in $(-\infty, 0)$ and $(1, \infty)$, leading to "probability leakage" and poor calibration near the boundaries.

### 1.5 Beta Regression Framework

To respect the support $\mathcal{Y} \in [0, 1]$, we model the predictive distribution using a **Beta distribution**:

$$p(y|\alpha, \beta) = \frac{y^{\alpha-1}(1-y)^{\beta-1}}{B(\alpha, \beta)}, \quad B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$$

**Head architecture.** Each Beta head is a two-layer perceptron rather than a bare linear read-out. Let $r \in \mathbb{R}^{L' \times d_{\text{in}}}$ denote the representation entering the head, where $d_{\text{in}}$ is its channel width. The head first maps $r$ through one hidden layer,

$$v = \sigma\big(W^{(1)}r + b^{(1)}\big) \in \mathbb{R}^{L' \times d_h},$$

where the superscript (1) indexes the first (and only) hidden layer: $W^{(1)} \in \mathbb{R}^{d_h \times d_{\text{in}}}$ is its weight matrix and $b^{(1)} \in \mathbb{R}^{d_h}$ its bias, $\sigma$ is the SiLU activation, and $d_h$ is the head width. The hidden activation $v$ is then read out by two separate linear layers, one per shape parameter:

$$z_\alpha = W^{(\alpha)}v + b^{(\alpha)}, \qquad z_\beta = W^{(\beta)}v + b^{(\beta)}, \qquad W^{(\alpha)}, W^{(\beta)} \in \mathbb{R}^{1 \times d_h}$$

Both read-outs consume the *same* $v$: the hidden layer extracts one shared intermediate feature from which the two shape parameters are then predicted independently. This lets $\alpha$ and $\beta$ draw on common structure - both describe the same predictive distribution - while still being free to move in opposite directions, which is what allows the head to represent "high mean, low variance" and "high mean, high variance" as distinct outputs.

The network outputs the shape parameters as $\alpha = \text{softplus}(z_\alpha) + 1$ and $\beta = \text{softplus}(z_\beta) + 1$. The softplus, $\ln(1+e^z)$, maps any real number to a positive one, which is all a Beta distribution formally requires ($\alpha, \beta > 0$). The added 1 goes further: since softplus is strictly positive, it guarantees $\alpha, \beta > 1$.

The reason is the shape of the Beta density on either side of 1. With $\alpha, \beta > 1$ the density is zero at both ends of $[0, 1]$ and has a single peak inside, at $(\alpha-1)/(\alpha+\beta-2)$; it says "engagement is probably around here", which is the kind of answer we want, and its mean and variance describe a best guess and the spread around it. If $\alpha < 1$, the density instead grows without bound as $y \to 0$ (and likewise at 1 if $\beta < 1$); with both below 1 it is U-shaped, piling its probability onto the two extremes and making the middle of the scale the *least* likely region. Such a prediction is useless as a point estimate: its mean $\alpha/(\alpha+\beta)$ lies exactly where the distribution itself expects the target least, and its variance no longer measures the spread around a single best guess - yet that is how both $\hat{\mu}$ and $\text{Var}[y]$ are used below and in test-time adaptation. The unbounded densities would also be an easy way to gain likelihood: some annotations lie very close to 0 or 1 (see target clipping below), and on those frames a model could raise the likelihood without limit by pushing $\alpha$ or $\beta$ below 1. The offset rules all of this out by construction. As a side effect, $\alpha + \beta > 2$ caps the predictive variance at $\mu(1-\mu)/3 \leq 1/12$, the variance of the uniform distribution on $[0, 1]$: the least certain prediction the model can make is "every engagement level is equally likely".

The predictive mean $\hat{\mu}$ and aleatoric uncertainty $\text{Var}[y]$ are:

$$\hat{\mu} = \frac{\alpha}{\alpha+\beta}, \quad \text{Var}[y] = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

The Beta variance is naturally boundary-aware, satisfying:

$$\lim_{\hat{\mu}\to 0} \mathrm{Var}[y] = 0 \quad \text{and} \quad \lim_{\hat{\mu}\to 1} \mathrm{Var}[y] = 0$$

This provides a principled measure of confidence that scales with the proximity to the interval limits.

**Target clipping.** Taking the logarithm of the Beta density above gives

$$\ln p(y \mid \alpha, \beta) = (\alpha - 1)\ln y + (\beta - 1)\ln(1 - y) - \ln B(\alpha, \beta),$$

so the log-likelihood contains the terms $\ln y$ and $\ln(1-y)$ explicitly. Both diverge at the endpoints of the support: $\ln y \to -\infty$ as $y \to 0^+$, and $\ln(1-y) \to -\infty$ as $y \to 1^-$. The engagement annotations do attain exactly 0 and 1, at which points the expression is undefined and evaluates to `-inf` or `NaN` in floating-point arithmetic - a single such frame is enough to corrupt the gradient of the entire batch. Targets are therefore clipped to $[\varepsilon,\ 1-\varepsilon]$ with $\varepsilon = 10^{-6}$ before the likelihood is evaluated. This is a numerical safeguard only: it moves an affected target by $10^{-6}$ on a scale of $[0, 1]$, which is four orders of magnitude below the resolution at which human annotators can meaningfully distinguish engagement levels.

### 1.6 Test-Time Adaptation (TTA) and Adaptive Thresholds

Following the differential-entropy principle of the original BiMamba-TTA paper [1], we replace Shannon entropy with a regression-compatible uncertainty proxy.

For the Beta predictive distribution $p(y \mid x) = \mathrm{Beta}(\hat{\alpha}(x), \hat{\beta}(x))$ that respects the support $y \in [0, 1]$, we define

$$U(x) := \mathrm{Var}[y] = \frac{\hat{\alpha}(x)\hat{\beta}(x)}{(\hat{\alpha}(x) + \hat{\beta}(x))^2(\hat{\alpha}(x) + \hat{\beta}(x) + 1)}$$

**What $U(x)$ is, intuitively.** In the original BiMamba-TTA formulation [1], $U(x)$ is the model's own admission of how unsure it is about the sample $x$. Test data carries no labels, so the model cannot know which of its predictions are correct; $U(x)$ is the next best thing, a self-assessment used to decide which unlabelled windows are trustworthy enough to learn from. Low $U(x)$ means the predictive distribution is sharply peaked - the model is committing to an answer - and such frames make safe pseudo-labels. High $U(x)$ means the distribution is spread out and the prediction is close to a guess; adapting on those would teach the model its own noise.

**Why variance.** For classification, $U(x)$ is the Shannon entropy of the softmax output, a direct measure of how spread the predicted distribution is. Regression has no softmax, so the natural generalisation is the differential entropy $H(p)$ of the predictive density - but for a Beta distribution $H(p)$ requires the log-Beta function and two digamma evaluations per frame, which is the transcendental cost we are trying to avoid in a real-time streaming filter.

The way out is to notice what the filter actually does with $U(x)$. It never uses the value of $U(x)$ on its own; it only asks whether a window's uncertainty is higher or lower than that of the other windows in the same batch (below). Any measure that puts the windows in the same order - from most to least uncertain - therefore selects exactly the same windows as $H(p)$, and we are free to choose the cheapest such measure. This reframes the problem from "compute the entropy" to "find the cheapest quantity that ranks windows the same way", and the predictive variance is the obvious first candidate: it is the standard measure of spread, it is already available in closed form from the shape parameters the network emits, and it costs four arithmetic operations. The remainder of this subsection examines how far it does rank them the same way.

### 1.6.1 Why Predictive Variance is a Monotonic Proxy for Differential Entropy at a Fixed Mean

This choice is justified because variance is a *strictly monotonic* function of the differential entropy $H(p)$ of the Beta distribution. To see this explicitly, re-parameterise the Beta distribution by its mean and concentration:

$$\mu = \frac{\hat{\alpha}}{\hat{\alpha} + \hat{\beta}}, \qquad \kappa = \hat{\alpha} + \hat{\beta}.$$

Substituting these parameters yields the simple expression

$$U(x) = \frac{\mu(1-\mu)}{\kappa + 1}.$$

For any fixed mean $\mu \in (0, 1)$, the partial derivative

$$\frac{\partial U}{\partial \kappa} = -\frac{\mu(1-\mu)}{(\kappa+1)^2} < 0$$

shows that variance is strictly decreasing in $\kappa$: larger concentration $\kappa$ produces a tighter distribution and therefore smaller variance.

The differential entropy of the Beta distribution has the closed-form expression

$$H(p) = \ln B(\hat{\alpha}, \hat{\beta}) - (\hat{\alpha} - 1)\psi(\hat{\alpha}) - (\hat{\beta} - 1)\psi(\hat{\beta}) + (\hat{\alpha} + \hat{\beta} - 2)\psi(\hat{\alpha} + \hat{\beta}),$$

where $B(\cdot, \cdot)$ is the Beta function and $\psi(\cdot)$ is the digamma function. Substituting the same re-parameterisation $\hat{\alpha} = \mu\kappa$, $\hat{\beta} = (1-\mu)\kappa$ and taking the derivative with respect to $\kappa$ likewise yields

$$\frac{\partial H}{\partial \kappa} < 0$$

for any fixed $\mu \in (0, 1)$. Thus both $U(x)$ and $H(p)$ are strictly decreasing functions of $\kappa$: larger concentration (tighter distribution) produces both smaller variance *and* smaller differential entropy. Consequently, for predictions with the same mean, ranking by variance and ranking by differential entropy give exactly the same order.

The argument holds only at a fixed mean. Across different means the two measures can disagree: Beta$(1.5, 7)$ (mean 0.18) has the larger variance, 0.0153 against 0.0122, but the smaller entropy, $-0.825$ against $-0.787$, compared with Beta$(9.5, 10)$ (mean 0.49). Among randomly drawn pairs of Beta distributions with $\alpha, \beta > 1$, about 5% are ordered differently by the two measures. Variance is therefore a cheap proxy that agrees with differential entropy on how concentrated a prediction is at a given engagement level, not an exact substitute for it.

Variance is numerically stable because its formula consists only of basic arithmetic operations (no logarithms, no digamma or Gamma functions, and no risk of underflow when $\hat{\alpha}$ or $\hat{\beta}$ become large). It is boundary-aware: $\lim_{\mu \to 0^+} U = 0$ and $\lim_{\mu \to 1^-} U = 0$, which matches the intuition that a prediction that is certain to be near the extremes of engagement (fully disengaged or fully engaged) should carry low uncertainty. Finally, variance is already required for the Beta negative log-likelihood (Beta NLL) loss used during training:

$$\mathcal{L}_{\text{NLL}}(x, y) = -[(\hat{\alpha} - 1)\ln y + (\hat{\beta} - 1)\ln(1 - y) - \ln B(\hat{\alpha}, \hat{\beta})].$$

Because the variance appears directly in the predictive distribution (and therefore in every forward pass), it incurs no extra computation at inference time. All these properties make variance the natural, cheap uncertainty proxy for regression.

A sample is selected for the TTA backward pass if the multimodal prediction is confident (low uncertainty) but the unimodal branches are uncertain (high uncertainty), i.e.,

$$S(x) = \mathbb{I}\Big\{U_{\text{multi}}(x) < \gamma_m \wedge \frac{1}{M}\sum_{i=1}^{M} U_i(x) > \gamma_u\Big\}.$$

Here $x$ is one window, and every $U$ is the per-frame predictive variance averaged over the window's frames; $U_{\text{multi}}$ is additionally averaged over the two role heads, and the unimodal average runs over all $M$ role-feature streams with equal weights.

To handle non-stationary streaming data (i.e., test-time engagement signals that arrive continuously and whose statistical distribution drifts over time due to fatigue, cultural differences, sensor drift, etc.), the thresholds are not fixed but recomputed for every inference batch $\mathcal{W}$ of $B_{\text{inf}}$ consecutive windows from one session:

$$\gamma_m = \text{Quantile}_{q_m}\Big(\{U_{\text{multi}}(x)\}_{x\in\mathcal{W}}\Big), \qquad \gamma_u = \text{Quantile}_{q_u}\Big(\Big\{\tfrac{1}{M}\textstyle\sum_i U_i(x)\Big\}_{x\in\mathcal{W}}\Big),$$

with $q_m = 0.3$, $q_u = 0.7$ and $B_{\text{inf}} = 16$. $\text{Quantile}_q$ of a set of numbers is the value below which a fraction $q$ of them lies. With $q_m = 0.3$, $\gamma_m$ is the 30th percentile of the fused-head uncertainties in the batch, so the first condition, $U_{\text{multi}}(x) < \gamma_m$, keeps the 30% of windows on which the multimodal head is *most* confident (five of the 16). With $q_u = 0.7$, $\gamma_u$ is the 70th percentile of the averaged unimodal uncertainties, so the second condition keeps the 30% of windows on which the unimodal heads are *least* confident. A window is used for adaptation only if it belongs to both groups. If the two uncertainties were unrelated, about $0.3 \times 0.3 = 9\%$ of the windows would qualify; in practice it was 2 to 6% (Section 4).

The thresholds are computed afresh for every batch, from that batch's own $B_{\text{inf}}$ windows rather than from a running history. With 10 s windows and a 5 s window hop, one batch covers about 85 s of the interaction, so the thresholds follow drift within a session while selection and update run as one batched operation. Here "non-stationary streaming data" means the incoming test distribution is not fixed but changes gradually; recomputing the thresholds on every batch tracks this drift. The batch must contain several windows: batches with fewer than four windows (the last, partial batch of a session) are left unadapted.

The quantile levels make the thresholds *relative rather than absolute*. A fixed cutoff such as $U(x) \leq 0.02$ would be meaningless across domains: a model that is uniformly less certain on Japanese sessions than on English ones would have every frame rejected on one corpus and every frame accepted on the other, purely because its overall uncertainty level shifted. By setting the thresholds to quantiles of the uncertainties in the current batch, "confident" is redefined at each moment to mean *confident by the standards of what this model is currently producing on this stream*, and the filter keeps selecting a stable fraction of windows regardless of the absolute uncertainty level. Percentiles are used rather than the mean and standard deviation because they are order statistics and therefore insensitive to the occasional extreme value - a few corrupted frames shift a percentile barely at all, whereas they can move a mean arbitrarily far.

### 1.7 Uncertainty Estimation Methods

Uncertainty is formally quantified as the variance of the predictive distribution. Two kinds are usually distinguished. *Aleatoric* uncertainty is noise in the data itself: annotators disagree, behaviour is ambiguous, a face is partly out of frame. More training data would not remove it. *Epistemic* uncertainty is the model's own lack of knowledge: it is large on inputs unlike those seen in training - for example, recordings from a different corpus or culture - and shrinks as the model sees more data. We describe three methods. They do *not* each capture both kinds: the

first measures only aleatoric uncertainty, the second only epistemic uncertainty, and the third is their sum, so both are covered only by the third method, or by the first two together. All three are expressed in terms of the Beta predictive distribution $p(y \mid x) = \mathrm{Beta}(\hat{\alpha}(x), \hat{\beta}(x))$. Of these, only the first is realised in the current implementation; methods 2 and 3 require stochastic regularisation that the present architecture does not contain (see the remark at the end of this subsection).

1. **Parametric Beta (Aleatoric Uncertainty):** This is the direct predictive variance obtained from the dual-head Beta output:

$$U(x) = \mathrm{Var}[y] = \frac{\hat{\alpha}(x)\hat{\beta}(x)}{(\hat{\alpha}(x) + \hat{\beta}(x))^2(\hat{\alpha}(x) + \hat{\beta}(x) + 1)}.$$

   Because the variance is computed analytically from the learned shape parameters, it is the fastest and default choice for the TTA filter, and it is the quantity actually used by the implemented selection rule $S(x)$. It measures only aleatoric uncertainty: a single network with fixed weights can express how noisy it expects the target to be for a given input, but it has no means of expressing doubt about its own weights, and on an input unlike anything seen in training it can be confidently wrong.

2. **Monte Carlo Dropout (Epistemic Uncertainty):** We keep dropout active at inference and draw $T$ stochastic forward passes (typically $T = 10$–$20$). For each pass $t$ we obtain a Beta distribution with parameters $(\hat{\alpha}^{(t)}, \hat{\beta}^{(t)})$ and corresponding mean $\hat{\mu}^{(t)} = \dfrac{\hat{\alpha}^{(t)}}{\hat{\alpha}^{(t)} + \hat{\beta}^{(t)}}$. The epistemic uncertainty is the variance of these means:

$$U_{\mathrm{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}^{(t)}(x) - \bar{\mu}(x)\big)^2,$$

   where $\bar{\mu}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{\mu}^{(t)}(x)$. This term quantifies the model's "lack of knowledge" (how sensitive the prediction is to random weight masks) and is especially useful under distribution shift (NoXi $\rightarrow$ NoXi+J). It measures only epistemic uncertainty: it looks at how much the means of the $T$ passes disagree with each other and ignores how wide each pass's own distribution is.

3. **Total Uncertainty (Hybrid):** Using the law of total variance, the full predictive uncertainty decomposes as

$$U_{\mathrm{total}}(x) = \underbrace{\frac{1}{T}\sum_{t=1}^{T}\mathrm{Var}\big[\mathrm{Beta}(\hat{\alpha}^{(t)}, \hat{\beta}^{(t)})\big]}_{\text{aleatoric (data)}} + \underbrace{\frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}^{(t)}(x) - \bar{\mu}(x)\big)^2}_{\text{epistemic (model)}}.$$

   It captures both kinds because the law of total variance splits the variance of the mixture of the $T$ predicted distributions exactly into these two parts. The decomposition is mainly used as a diagnostic to monitor whether the shift is mostly epistemic.

In the implemented filter, $U(x)$ is always the first quantity, the parametric Beta variance. The law of total variance guarantees that $U_{\mathrm{total}}$ is the correct variance of the mixture distribution induced by MC dropout.

**Implementation status.** The current architecture contains no dropout layers, so methods 2 and 3 cannot be evaluated as written: with deterministic weights, every one of the $T$ forward passes returns the identical Beta distribution and $U_{\mathrm{epi}} \equiv 0$. Realising them requires inserting dropout into the Beta heads (and optionally the BiMamba projections) and retaining it at

inference. Until then, methods 2 and 3 should be read as a planned extension rather than a component of the reported system. They are deferred: adaptation is first re-tested on the data of a single participant (Section 4.8), and only if it still does not improve the predictions there will they be implemented, probably starting directly with method 3.

### 1.8 Training Objective

The likelihood we train on and the metric we are scored by ask different questions. The Beta negative log-likelihood looks at one frame at a time and asks whether the predicted distribution is right there. CCC looks at all frames together and asks whether the predictions rise and fall in the same pattern as the targets. A model can therefore place a sensible distribution on every frame and still order the frames badly. Take two frames whose true engagement is 0.55 and 0.60, and a model that predicts them the wrong way round, 0.60 and 0.55, each with a moderately wide distribution (concentration $\kappa = 10$, a standard deviation of about 0.15). Each prediction is only 0.05 away from its target, well inside its own spread, so the likelihood barely notices: the summed negative log-likelihood of the two frames rises by only about 0.1 nats compared with the correct predictions. Yet the model now claims that the second frame is *less* engaged than the first, which is precisely the kind of error CCC penalises. Over a whole session, a model whose predictions stay close to the true level but wobble in directions unrelated to the true ups and downs can achieve a good likelihood and a poor correlation at the same time. We therefore add three terms to the likelihood,

$$\mathcal{L} = \underbrace{\frac{1}{|\mathcal{R}|}\sum_{r\in\mathcal{R}} \mathcal{L}^{w}_{\mathrm{NLL}}\big(\alpha^r, \beta^r, y^r\big)}_{\text{per-frame fit}} + \underbrace{\lambda_c\big(1-\widehat{\mathrm{CCC}}\big)}_{\text{ordering}} + \underbrace{\frac{\lambda_u}{M}\sum_{i=1}^{M} \mathcal{L}^{w}_{\mathrm{NLL}}\big(\alpha_i, \beta_i, y^{r(i)}\big)}_{\text{per-modality supervision}} + \underbrace{\lambda_f\, \mathcal{L}_{\mathrm{fair}}}_{\text{fairness}},$$

where $r(i)$ denotes the role of stream $i$, and $\lambda_c, \lambda_u, \lambda_f \geq 0$ set how much each term counts.

**Reweighting the likelihood.** Describe a Beta density by its mean $\mu = \alpha/(\alpha+\beta)$ and its precision $\kappa = \alpha + \beta$, so $\alpha = \mu\kappa$ and $\beta = (1-\mu)\kappa$; large $\kappa$ means a narrow, confident distribution. Differentiating $\ln p(y) = (\alpha-1)\ln y + (\beta-1)\ln(1-y) - \ln B(\alpha,\beta)$ with respect to $\mu$, with $\partial\alpha/\partial\mu = \kappa$ and $\partial\beta/\partial\mu = -\kappa$, and using $\ln B(\alpha,\beta) = \ln\Gamma(\alpha) + \ln\Gamma(\beta) - \ln\Gamma(\alpha+\beta)$ so that $\partial \ln B/\partial\alpha = \psi(\alpha) - \psi(\alpha+\beta)$ and $\partial \ln B/\partial\beta = \psi(\beta) - \psi(\alpha+\beta)$, the $\psi(\alpha+\beta)$ terms cancel and leave

$$\frac{\partial\big(-\ln p\big)}{\partial\mu} = \kappa\left[\ln\frac{1-y}{y} + \psi(\mu\kappa) - \psi\big((1-\mu)\kappa\big)\right] \;\propto\; \kappa.$$

The correction the mean receives is thus multiplied by the model's own confidence, so frames it is already sure about push hardest and frames it is unsure about barely move it - the opposite of what we want, since the uncertain frames are the ones with something left to learn. This is a known failure of likelihood training with predicted uncertainty [21]. We fix it by scaling each frame's loss by $w_t = \kappa_t^{-\beta_w}$, giving $\mathcal{L}^{w}_{\mathrm{NLL}} = \sum_t w_t \mathcal{L}_{\mathrm{NLL},t} / \sum_t w_t$ and a correction proportional to $\kappa^{1-\beta_w}$; $\beta_w = 0$ leaves the likelihood untouched and $\beta_w = 1$ removes the confidence factor completely, and we use $\beta_w = 0.5$. Crucially $w_t$ is computed from the current $\alpha_t, \beta_t$ but treated as a fixed number when derivatives are taken, so no gradient flows back through it. Were it differentiable, $w_t$ would become part of the objective: the model could then lower the loss by shifting its confidence so as to shrink the weight, rather than by predicting better, and the best $\kappa$ would no longer be the one the likelihood asks for. Treated as a constant, $w_t$ changes only how strongly each frame pulls, not where the pull ends - any point at which every frame's likelihood gradient vanishes is still a stationary point of the weighted loss. In practice $w_t$ is recomputed from the current prediction at every training step and discarded after the update; it is never learned. The ablation of Section 4 measures what the reweighting contributes.

**Ordering term.** $\widehat{\mathrm{CCC}}$ is the sample concordance coefficient of Section 3.1 computed on the predicted means and the targets of the current minibatch, pooled across both roles and all frames in it. Every ingredient - the two means, the two variances, the covariance - is a smooth function of the predictions, so the term can be differentiated, and since $\mathrm{CCC} = \rho\, C_b$, minimising $1 - \widehat{\mathrm{CCC}}$ pushes on exactly the factor the likelihood leaves loose. Two limitations follow from estimating it on a batch. It is biased when the batch is small, because the same frames supply both the moments and the ratio built from them; and it reflects the pooled quantity we are scored on only if the batch spans several sessions, since a batch from one session measures ordering within that session alone. Batches are accordingly assembled from windows drawn at random from all sessions of the training split; Section 4.3 shows how much this matters.

**Per-modality supervision.** Each auxiliary head is trained on the target of the role whose stream it reads. This keeps every modality branch predictive in its own right instead of drifting into whatever representation happens to suit the fusion stage, and it supplies the calibrated per-modality distributions that test-time adaptation (Section 1.6) compares against each other. Dividing by $M$ keeps this contribution from growing with the number of streams: at $\lambda_u = 0.5$ and $M$ streams the undivided sum would outweigh the multimodal term by $\lambda_u M$, which for the final configuration ($M = 10$) is a factor of five - so most of the gradient would come from heads that are thrown away at evaluation time.

**Fairness term.** The Conditional Demographic Disparity of Section 3.2 is built from differences of group averages inside strata of the ground truth, and every step is differentiable, so it can be trained against rather than only measured. Splitting the batch into $n_b$ quantile bins $\mathcal{B}_1, \ldots, \mathcal{B}_{n_b}$ of the target, and writing $\overline{\hat{\mu}}_b^{\,A}$ and $\overline{\hat{\mu}}_b^{\,B}$ for each group's mean prediction inside bin $b$, we penalise

$$\mathcal{L}_{\mathrm{fair}} = \frac{\sum_{b=1}^{n_b} |\mathcal{B}_b| \left(\overline{\hat{\mu}}_b^{\,A} - \overline{\hat{\mu}}_b^{\,B}\right)^2}{\sum_{b=1}^{n_b} |\mathcal{B}_b|},$$

with bins missing either group left out of both sums. Squaring removes the sign, so the term is smallest exactly when the two groups get the same prediction at the same true engagement, and weighting by bin size matches how the metric is estimated at evaluation. Note that the penalty is identically zero on a batch containing only one group, so batches must mix groups for it to do anything. With 8 windows per batch this is frequently violated at the level of individual bins, and in the ablations of Section 4 the term has no measurable effect on $\mathrm{CDD}_G$ at $\lambda_f = 0.05$.

### 1.9 Test-Time Loss Derivations

The TTA objective enforces consistency between the unimodal student heads and the multimodal teacher while still supervising the students toward the multimodal prediction. We define the **Mutual Information Sharing (MIS) loss** as the KL divergence that pushes each unimodal Beta distribution toward the multimodal one:

$$\mathcal{L}_{\mathrm{mis}} = \sum_{i=1}^{M} D_{KL}\big(\mathrm{Beta}(\alpha_i, \beta_i) \parallel \mathrm{Beta}(\alpha_{\mathrm{multi}}, \beta_{\mathrm{multi}})\big),$$

where the closed-form Beta KL divergence, with $B_i := B(\alpha_i, \beta_i)$, is:

$$D_{\mathrm{KL}}(B_1 \parallel B_2) = \ln \frac{B(\alpha_2, \beta_2)}{B(\alpha_1, \beta_1)} + (\alpha_1 - \alpha_2)\psi(\alpha_1) + (\beta_1 - \beta_2)\psi(\beta_1) + (\alpha_2 - \alpha_1 + \beta_2 - \beta_1)\psi(\alpha_1 + \beta_1) \quad (1)$$

The final TTA loss combines the MIS consistency term with a supervision term that treats the multimodal mean as a pseudo-target for each unimodal head:

$$\mathcal{L}_{TTA} = \mathcal{L}_{\mathrm{mis}} + \lambda \sum_{i=1}^{M} \mathrm{NLL}_{\mathrm{Beta}}\big(\hat{\mu}_{\mathrm{multi}},\, \alpha_i, \beta_i\big).$$

Here $\lambda > 0$ is a hyperparameter that controls the relative importance of the two terms: larger $\lambda$ emphasises direct supervision from the multimodal teacher, while smaller $\lambda$ puts more weight on cross-modal consistency. The term $\mathrm{NLL}_{\mathrm{Beta}}(\hat{\mu}_{\mathrm{multi}}, \alpha_i, \beta_i)$ is the Beta negative log-likelihood evaluated at the multimodal mean $\hat{\mu}_{\mathrm{multi}}$ (treated as a pseudo-label) with the unimodal shape parameters.

**Implementation.** Each role's multimodal head teaches only the unimodal heads that read that role's streams, and both loss terms are averaged over the frames of the *selected* windows only,

$$\mathcal{L}_{TTA}(\mathcal{W}) = \frac{\sum_{x \in \mathcal{W}} S(x)\, \ell_{TTA}(x)}{\sum_{x \in \mathcal{W}} S(x)},$$

where $\ell_{TTA}(x)$ is the loss above evaluated on window $x$; windows with $S(x) = 0$ contribute nothing. The pseudo-label $\hat{\mu}_{\mathrm{multi}}$ is treated as a constant (no gradient flows into the teacher through it), whereas the MIS term acts on both heads. Gumbel noise is switched off during adaptation, and batch normalisation uses the statistics of the current batch for the update step. After the update, the predictions for the batch are recomputed in evaluation mode with the adapted weights, so every output reflects the adaptation it triggered. Adaptation is episodic: parameters and optimiser state are reset to the trained model at the start of every session. The update is a single Adam step with learning rate $\eta_{\mathrm{TTA}}$ per batch.

### 1.10 Surgical Fine-Tuning and Variance Starvation

We only update specific surgical layers to prevent catastrophic forgetting:

- **BatchNorm layers:** These encode per-channel activation statistics, allowing for immediate adaptation to domain shifts. In EngageNet these are the BN stages inside every InitEncoder.
- **First convolution in InitEncoder:** Re-calibrates low-level feature extraction (e.g., sensor noise) before temporal/modal mixing.
- **First FC layer in Inter-modal BiMamba:** Recalibrates cross-modal fusion (e.g., adjusting the weight of audio vs. visual cues for different cultures). Concretely this is the first dense projection inside the cross-modal BiMamba block.

All remaining parameters receive exactly zero update. The adaptation uses its own optimiser for this purpose - Adam on the surgical layers and an identically zero update elsewhere - because masking only the gradients under the training optimiser would still let its weight decay move the frozen weights.

The word *surgical* refers to the choice of layers, not to their size. In the core configuration the listed layers hold 1.20M of the model's 2.06M parameters, almost all of them in the first convolutions of the W2v-BERT ($1024 \to 128$ channels, kernel 5) and OpenFace2 ($714 \to 128$) encoders, so by parameter count the adaptation touches the majority of the network.

This surgical approach mitigates the **Variance Starvation Problem (VSP)**. In VSP, a model minimises the Beta NLL by artificially inflating $\mathrm{Var}[y]$ (i.e., reducing the concentration $\kappa = \hat{\alpha} + \hat{\beta}$) on hard samples; the gradient to the mean head then vanishes because the loss term becomes insensitive to prediction error. By freezing the majority of the backbone and updating only the listed surgical layers, we force the model to improve genuine feature alignment rather than using uncertainty as a "slack variable" to bypass difficult data.

## 1.11 Architectural Justification for the Beta Regression Framework

To model bounded continuous engagement levels $y \in [0, 1]$ alongside reliable predictive uncertainty, several alternative statistical formulations could be considered: the Kumaraswamy distribution, logit-normal regression, and distribution-free conformalized regression. In this section, we mathematically demonstrate why the Beta distribution is uniquely suited to meet the real-time constraints, closed-form requirement, and test-time adaptation (TTA) mechanics of EngageNet.

### Comparison with the Kumaraswamy Distribution

The Kumaraswamy distribution [8] is often preferred over the Beta distribution in standard deep learning tasks [9] because its probability density function and cumulative distribution function rely strictly on elementary algebraic powers, avoiding transcendental functions. For shape parameters $a, b > 0$, its CDF is $F(x; a, b) = 1 - (1 - x^a)^b$. This formulation simplifies the calculation of the Negative Log-Likelihood (NLL) during training and avoids computing digamma and trigamma functions during backpropagation, circumventing the gradient instabilities that cause the Variance Starvation Problem (VSP).

However, EngageNet's TTA selection filter $S(x)$ evaluates the predictive variance $U(x)$ at *inference time* on every incoming frame of a non-stationary data stream. The $n$-th raw moment of a Kumaraswamy random variable $Y$ is given by:

$$\mathbb{E}[Y^n] = b \cdot B\left(1 + \frac{n}{a}, b\right) = \frac{b\,\Gamma(1 + n/a)\,\Gamma(b)}{\Gamma(1 + n/a + b)}$$

where $B(\cdot, \cdot)$ is the transcendental Beta function and $\Gamma(\cdot)$ is the Gamma function. Consequently, the predictive variance of the Kumaraswamy distribution is expressed as:

$$\mathrm{Var}[Y] = b \cdot B\left(1 + \frac{2}{a}, b\right) - \left[b \cdot B\left(1 + \frac{1}{a}, b\right)\right]^2$$

If EngageNet adopted the Kumaraswamy distribution, the model would be forced to compute multiple expensive Gamma and Beta functions during real-time streaming inference to evaluate the TTA filter. In contrast, the Beta distribution shifts this transcendental computational burden entirely to the offline training phase via the polygamma functions in the backward pass. Its inference-time predictive variance $U(x)$ is computed via lightweight, pure algebraic operations:

$$U(x) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

Furthermore, proving a strictly monotonic relationship between the variance $\mathrm{Var}[Y]$ and the differential entropy $H(Y)$ under arbitrary parameter updates is mathematically intractable for the Kumaraswamy distribution, whereas the Beta distribution cleanly satisfies $\frac{\partial U}{\partial \kappa} < 0$ and $\frac{\partial H}{\partial \kappa} < 0$ under the precision mapping $\kappa = \alpha + \beta$.

### Comparison with Logit-Normal Regression

Logit-normal regression maps the bounded domain $y \in [0, 1]$ to an unbounded latent domain $z \in \mathbb{R}$ using the logit transformation, assuming the latent variable follows a Gaussian distribution:

$$z = \mathrm{logit}(y) = \ln\left(\frac{y}{1 - y}\right) \sim \mathcal{N}(\mu, \sigma^2)$$

While the latent Gaussian log-likelihood is highly stable to optimize during training, the moments of the resulting logit-normal variable do not exist in closed form. The true predictive

mean and variance must be resolved via numerical integration:

$$\mathbb{E}[Y] = \int_0^1 \frac{1}{y(1-y)\sigma\sqrt{2\pi}} \exp\left(-\frac{(\text{logit}(y)-\mu)^2}{2\sigma^2}\right) \cdot y \, dy$$

Performing numerical integration over $N$ categories across multiple unimodal branches for every incoming frame introduces an insurmountable latency bottleneck during real-time test-time adaptation.

Crucially, EngageNet utilizes an inter-modal information sharing loss ($\mathcal{L}_{mis}$) that minimizes the Kullback-Leibler (KL) divergence between the unimodal predictions and the fused network prediction. While computing the KL divergence between two latent Gaussians in logit space is trivial, minimizing it optimizes a distorted geometry that stretches gradients to infinity near the boundaries (0 and 1). Computing the exact KL divergence between two true logit-normal densities within the probability domain $[0, 1]$ lacks an analytical closed-form expression [7]. Because the Beta distribution possesses a known analytical KL divergence 1, it provides a stable, exact, and geometry-preserving target for cross-modal alignment.

**Comparison with Conformalized Regression**

Conformalized regression (or Conformal Prediction) [10] offers a distribution-free, non-parametric framework that provides a valid prediction interval $[\hat{y}-q, \hat{y}+q]$ satisfying a user-defined coverage guarantee $1-\alpha$. While mathematically robust, conformal prediction fundamentally requires the data stream to satisfy the property of exchangeability.

In streaming test-time adaptation settings, the user exhibits non-stationary distribution shifts (e.g., sensor degradation, fluctuating lighting conditions, or progressive fatigue). This explicitly violates the exchangeability assumption, causing the empirical quantile threshold $q$ computed on a static source calibration set to lose its strict coverage guarantees. While online adaptive variants (such as Adaptive Conformal Inference) [11] can dynamically scale the interval width based on sequential coverage errors, they operate as reactive heuristic controllers rather than generative models of the target density.

Most importantly, conformal prediction produces a discrete boundary region rather than a smooth, parametric probability density function. Because it is non-differentiable, it cannot supply a gradient operator to backpropagate local self-supervised likelihood updates ($\mathcal{L}_{TTA}$) during a single-frame backward pass. It also cannot participate in information-theoretic objectives like the mutual information sharing loss ($\mathcal{L}_{mis}$). Thus, while conformal regression is excellent for post-hoc uncertainty visualization, it cannot serve as the active functional engine that drives online weight updates within an adaptive neural network architecture.

# 2 Inter-modal Fusion with Learned Modality Ordering

## 2.1 The Order-Sensitivity Problem

Recall from Section 1.3.2 that the inter-modal BiMamba module treats modality identity as its sequence axis, processing at each time step the length-$M$ sequence

$$\big(u_1^{(t)}, u_2^{(t)}, \ldots, u_M^{(t)}\big), \qquad u_i^{(t)} \in \mathbb{R}^{C'},$$

with each modality contributing exactly one position.

The ordering of these positions - audio before video, or video before pose - is chosen arbitrarily by the practitioner; no prior knowledge assigns a canonical rank to distinct measurement modalities. We show that this arbitrariness introduces a structural bias, because selective state-space models are inherently order-sensitive.

Fix a time step $t$, let $s_k \in \mathbb{R}^{C'}$ denote the input at sequence position $k$ - the representation of the modality placed $k$-th - and let $\xi_k$ be the SSM hidden state after position $k$. The Mamba selective SSM [2] defines the recurrence

$$\xi_k = \bar{A}_k(s_k)\,\xi_{k-1} + \bar{B}_k(s_k)\,s_k, \qquad y_k = C_k(s_k)\,\xi_k,$$

where $\xi_0 = \mathbf{0}$ and $\bar{A}_k, \bar{B}_k, C_k$ are input-dependent operators instantiated via the selection mechanism.

**Proposition.** For any SSM with non-trivial state transition (i.e. $\bar{A} \neq I$), the output sequence is not invariant under permutations of the input.

*Proof.* It suffices to consider $M = 2$ and a linear, non-selective SSM with constant operators $\bar{A}, \bar{B}, C$. Under the ordering $(u_1, u_2)$:

$$\xi_1 = \bar{B}\,u_1, \qquad y_2 = C\big(\bar{A}\bar{B}\,u_1 + \bar{B}\,u_2\big)\,.$$

Under the swapped ordering $(u_2, u_1)$:

$$\xi_1' = \bar{B}\,u_2, \qquad y_2' = C\big(\bar{A}\bar{B}\,u_2 + \bar{B}\,u_1\big)\,.$$

The difference is

$$y_2 - y_2' = C\bar{B}(\bar{A} - I)(u_1 - u_2).$$

This vanishes for all $u_1 \neq u_2$ only if $\bar{A} = I$, which degenerates the SSM to a memoryless map and eliminates all temporal memory - contradicting the purpose of state-space modelling. In the selective Mamba SSM, the input-dependent operators $\bar{A}_k(s_k)$ and $\bar{B}_k(s_k)$ introduce additional asymmetric cross-terms that further break permutation symmetry. □

Bidirectionality alone does not resolve the problem. In the inter-modal block the backward pass reverses the modality sequence, so the backward SSM processes the modalities in the order $M, M-1, \ldots, 1$; each modality's backward hidden state therefore accumulates context from those that follow it in the reversed sequence. The averaged representation

$$H = \frac{h^{\rightarrow} + h^{\leftarrow}}{2}$$

remains order-dependent: both the forward and backward context seen by modality $i$ change qualitatively with its position in the arbitrary input order, so the same physical modality receives a structurally different representation depending solely on where it was placed in that order.

## 2.2 Differentiable Modality Ordering via Gumbel-Sinkhorn

We replace the fixed concatenation order with a *learned, input-dependent permutation* computed differentiably via the Gumbel-Sinkhorn operator [3]. The key intuition is that the optimal ordering is itself a function of the data: for a sample in which audio is uninformative, placing it first in the forward SSM pass deposits unreliable context into the hidden states of all subsequent modalities, whereas placing it last confines the contamination to the final position.

### Uniform channel projection

Different InitEncoders naturally produce output representations of different widths: for instance, an audio encoder may output $C'_{\text{audio}} = 64$ channels while a video encoder outputs $C'_{\text{video}} = 256$ channels. These *heterogeneous dimensions* prevent treating the modality outputs as interchangeable objects of the same type, which is a prerequisite for learning a permutation over them. To resolve this, a modality-specific affine map $f_i : \mathbb{R}^{C'_i} \to \mathbb{R}^{C'}$, defined by $f_i(v) = W_i^{\text{proj}} v + b_i^{\text{proj}}$ with $W_i^{\text{proj}} \in \mathbb{R}^{C' \times C'_i}$ and $b_i^{\text{proj}} \in \mathbb{R}^{C'}$, is appended to each InitEncoder to

linearly project its output into a shared embedding space of dimension $C'$. After this projection, and after the intra-modal module that follows it, $u_i \in \mathbb{R}^{L' \times C'}$ for all $i \in \{1, \ldots, M\}$: all $M$ representations have the same shape, which is what allows them to be treated as interchangeable positions of one sequence and reordered. As with the encoders themselves, the projection is shared between the expert and novice streams of a given feature type.

**Score matrix**

A *meta-network* $\phi$ is a small auxiliary network whose sole purpose is to predict how modalities should be ordered, operating on top of the main feature representations without participating in the primary task of engagement prediction. Concretely, $\phi$ maps the set $\{u_i\}_{i=1}^{M}$ to an $M \times M$ assignment *score* matrix $Z \in \mathbb{R}^{M \times M}$, where the entry $Z_{ij}$ quantifies the fitness of placing modality $j$ at sequence position $i$ in the inter-modal BiMamba: a large value of $Z_{ij}$ means the meta-network believes modality $j$ should be processed at step $i$. Each modality representation is first summarised by temporal mean-pooling:

$$\bar{u}_i = \frac{1}{L'} \sum_{t=1}^{L'} u_i^{(t)} \in \mathbb{R}^{C'}.$$

A bias-free projection into a score-embedding space of dimension $d_s$ turns each summary into a key, and each *position* has its own learned query vector:

$$k_j = W_K \, \bar{u}_j \in \mathbb{R}^{d_s}, \qquad p_i \in \mathbb{R}^{d_s},$$

where $W_K \in \mathbb{R}^{d_s \times C'}$ is shared across modalities and $p_1, \ldots, p_M$ are free parameters. The score is the scaled dot product

$$Z_{ij} = \frac{p_i^\top k_j}{\sqrt{d_s}}.$$

Row $i$ of $Z$ therefore asks how well each stream fits position $i$, and column $j$ how well stream $j$ fits each position: rows and columns index different kinds of objects. Our first formulation used a query computed from the data, $q_i = W_Q \, \bar{u}_i$, so that $Z_{ij} = q_i^\top k_j$ compared stream $i$ with stream $j$ and contained no notion of position at all; it also made the result depend on how the streams are numbered (see below). The division by $\sqrt{d_s}$ keeps the scores at a magnitude independent of $d_s$, as in scaled dot-product attention. It matters here because $Z$ is next divided by the annealing temperature $\tau$: an unscaled score of large magnitude has the same effect as a small $\tau$ and drives the Sinkhorn output towards a hard assignment before the annealing schedule intends it to. Both changes are evaluated in Section 4.6.

**Gumbel perturbation**

The Gumbel distribution is classically used in extreme value theory [5] to model the asymptotic distribution of the maximum of a sequence of independent and identically distributed random variables. Its probability density function for location $\mu$ and scale $\beta$ is given by:

$$f(x) = \frac{1}{\beta} e^{-(x-\mu)/\beta} e^{-e^{-(x-\mu)/\beta}}$$

In deep learning, the Gumbel-Max trick [4] leverages this distribution to draw discrete samples from a categorical distribution in a fully differentiable manner. By adding independent Gumbel noise to unnormalized log-probabilities (scores), the deterministic argmax of these perturbed values becomes mathematically equivalent to drawing a sample from the true underlying multinomial distribution.

To generate standard Gumbel noise ($\mu = 0, \beta = 1$) programmatically, we apply inverse transform sampling. This technique maps a standard uniform random variable to a target distribution by passing it through the inverse of that target's cumulative distribution function (CDF). Given that the standard Gumbel CDF is $F(x) = e^{-e^{-x}}$, setting a uniform random variable $U \sim \text{Uniform}(0, 1)$ equal to $F(x)$ and solving explicitly for $x$ yields the inverse mapping $x = -\ln(-\ln U)$. In practice $U$ is drawn from the open interval $(\epsilon, 1 - \epsilon)$ with $\epsilon = 10^{-6}$, since the double logarithm diverges at both endpoints. Thus, to enable stochastic exploration of orderings during training, the score matrix $Z$ is perturbed with i.i.d. standard Gumbel noise and scaled by a temperature parameter $\tau > 0$:

$$\tilde{Z}_{ij} = \frac{Z_{ij} + G_{ij}}{\tau}, \qquad G_{ij} = -\ln(-\ln U_{ij}), \quad U_{ij} \overset{\text{i.i.d.}}{\sim} \text{Uniform}(0, 1).$$

The temperature $\tau$ is annealed over the course of training to progressively concentrate probability mass towards a single hard permutation. We adopt a standard exponential decay schedule, updating the temperature at each epoch $t$ via $\tau_t = \max(\tau_\infty, \tau_0 \cdot \gamma^t)$, where $\gamma \in (0, 1)$ represents the decay rate, starting from an initial exploration temperature $\tau_0$ down to a tight categorical approximation threshold $\tau_\infty$. Concrete values are listed in Section 5.

**Sinkhorn normalisation**

The **Birkhoff-von Neumann theorem** [6] establishes that the set of doubly-stochastic matrices (square matrices with non-negative real entries whose rows and columns each sum to exactly 1) constitutes a convex polytope whose extreme points are precisely the set of discrete permutation matrices. Geometrically, a convex polytope is a bounded, multi-dimensional solid defined by flat facets, and its extreme points represent its absolute vertices. Because any interior point of a convex polytope can be expressed uniquely as a convex combination (a weighted average) of its vertices, any doubly-stochastic matrix can be decomposed into a convex combination of hard permutation matrices. This structural property makes the Birkhoff polytope a mathematically principled continuous relaxation of discrete permutations.

A standard Softmax operator is insufficient in this context because it operates independently across a single dimension, ensuring only that the rows sum to 1 (yielding a row-stochastic matrix). In an inter-modal fusion framework where rows represent sequence positions and columns map to distinct modalities, a row-stochastic matrix introduces severe structural redundancies. It permits multiple sequence positions to concurrently assign high probability mass to the exact same dominant modality column (such as audio), which causes the network to attend to duplicate representations while completely suppressing and ignoring other available modalities.

Constraining both rows and columns to sum to 1 eliminates this issue by enforcing a strict continuous one-to-one assignment map. Following [3], the doubly-stochastic matrix $P$ is obtained by the iterative Sinkhorn-Knopp algorithm, which alternates row and column normalisation of $\exp(\tilde{Z})$ until convergence.

**Log-domain implementation.** We do *not* evaluate $\exp(\tilde{Z})$ explicitly. With real feature magnitudes the scores of our first, unscaled formulation reached $10^4$–$10^5$, and nothing bounds the scaled scores during training either, so $\exp(\tilde{Z})$ overflows to $+\infty$ in single precision; the subsequent row normalisation then computes $\infty/\infty$ and returns NaN, which propagates through the entire network and destroys training. We therefore carry out the alternating normalisation entirely in log-space.

The operation that replaces division is the *log-sum-exp*, defined for a finite set of reals $\{x_1, \ldots, x_n\}$ as

$$\operatorname*{logsumexp}_j x_j \;:=\; \log \sum_{j=1}^{n} \exp(x_j),$$

that is, the logarithm of the sum of the exponentials. Its role here follows from a single identity. If a set of quantities is held in logarithmic form, $x_j = \log p_j$, then normalising them to sum to one, $p_j \mapsto p_j / \sum_{j'} p_{j'}$, becomes in logarithms

$$\log\left(\frac{p_j}{\sum_{j'} p_{j'}}\right) = \log p_j - \log \sum_{j'} p_{j'} = x_j - \operatorname*{logsumexp}_{j'} x_{j'}.$$

A normalisation is therefore a *subtraction* in log-space, never a division, and the sums themselves are never formed. Crucially, log-sum-exp is evaluated by first factoring out the largest element, $m = \max_j x_j$,

$$\operatorname*{logsumexp}_{j} x_j = m + \log \sum_{j} \exp(x_j - m),$$

which is algebraically exact and guarantees every exponential argument is non-positive, so no term can exceed 1 and overflow cannot occur regardless of how large the $x_j$ become.

Writing $\log Q^{(k)}$ for the matrix obtained after the row pass of iteration $k$, and $\log P^{(k+1)}$ for the result after the subsequent column pass, the scheme is

$$\log P^{(0)} = \tilde{Z},$$

$$\log Q_{ij}^{(k)} = \log P_{ij}^{(k)} - \operatorname*{logsumexp}_{j'} \log P_{ij'}^{(k)}, \qquad \log P_{ij}^{(k+1)} = \log Q_{ij}^{(k)} - \operatorname*{logsumexp}_{i'} \log Q_{i'j}^{(k)},$$

for $k = 0, \ldots, n_{\text{iter}} - 1$, with $P = \exp\big(\log P^{(n_{\text{iter}})}\big)$ returned only at the end. The first update normalises each row (the inner index runs over columns $j'$), the second normalises each column (the inner index runs over rows $i'$); $Q^{(k)}$ is simply the intermediate that is row-stochastic but not yet column-stochastic, and it is discarded once the column pass consumes it. Alternating the two passes is what drives the matrix towards satisfying both constraints at once.

This is algebraically identical to Sinkhorn-Knopp on $\exp(\tilde{Z})$ - each step divides by a row or column sum, which by the identity above is a subtraction of its logarithm - but the largest intermediate quantity is $\max_j \tilde{Z}_{ij}$ rather than $\exp(\max_j \tilde{Z}_{ij})$. The exponential map is applied once, at the end, to values that are by construction non-positive log-probabilities, so the returned entries lie in $(0, 1]$ and the gradient operator $\partial P / \partial Z$ remains well-conditioned throughout optimisation.

The number of iterations matters. Sinkhorn-Knopp reaches a doubly-stochastic matrix only in the limit; after finitely many iterations the result satisfies exactly only the constraint of its last pass, here the columns. With $n_{\text{iter}} = 10$ and unscaled scores, the rows of the learned $P$ summed to between 0.6 and 2.0 (Section 4.6), so some positions received about two streams and others about half of one. We therefore use $n_{\text{iter}} = 50$ and check the row and column sums of the learned $P$ on the validation windows.

**Soft reordering**

With $P$ computed as above, the permuted representation for position $i$ is the $P$-weighted convex combination of all intra-modal outputs. Concretely, the $i$-th row of $P$, denoted $P_{i\cdot} = (P_{i1}, \ldots, P_{iM})$, defines a probability distribution over the $M$ modalities; since $P$ is doubly-stochastic, we have $P_{ij} \geq 0$ and $\sum_j P_{ij} = 1$ for every $i$. The soft assignment is then:

$$\tilde{u}_i = \sum_{j=1}^{M} P_{ij}\, u_j \in \mathbb{R}^{L' \times C'}, \qquad i \in \{1, \ldots, M\}.$$

The entry $P_{ij} \in [0, 1]$ is the soft probability that modality $j$ is assigned to position $i$ in the inter-modal sequence; when $P$ is close to a hard permutation matrix (as happens after temperature

annealing), exactly one $P_{ij}$ per row will be close to 1 and the rest close to 0, recovering a near-discrete assignment. At every time step $t$ the inter-modal BiMamba then processes the reordered sequence $(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)})$ instead of $(u_1^{(t)}, \ldots, u_M^{(t)})$; all equations of Section 1.3.2 remain unchanged under this substitution. Note that $P$ is computed per sample: the ordering is input-dependent, so different sessions may resolve to different modality orderings.

**Inference**

At test time, Gumbel noise is suppressed ($G_{ij} = 0$), so the permutation becomes a deterministic function of the input; the soft doubly-stochastic $P$ is used directly for the reordering. $P$ is never rounded to a hard permutation matrix, neither during training nor at inference. The model is run at the temperature of the epoch whose checkpoint is used, not at $\tau_\infty$: the checkpoint was trained and selected at that temperature, and a lower one would produce a sharper $P$ than the model has ever seen. In our first implementation inference always used $\tau_\infty$, although training stops early and the selected epoch $e$ has $\tau = 0.95^{\,e-1}$, for example 0.63 at epoch 10.

**Dependence on the input numbering**

With one query per position, renumbering the streams by a permutation $\rho$ renumbers only the columns of $Z$: $Z' = ZR^\top$, where $R$ is the permutation matrix of $\rho$. Sinkhorn normalisation commutes with this, so $P' = PR^\top$, and the reordered sequence $\tilde{u}'_i = \sum_j P'_{ij} u'_j = \sum_j P_{ij} u_j = \tilde{u}_i$ is exactly the same, for the soft $P$ and at every temperature: the inter-modal BiMamba receives the same physical order of streams however they are numbered. Our first, stream-query formulation lacked this property. There the query of row $i$ was computed from stream $i$, so a renumbering renumbered the rows as well, $Z' = RZR^\top$, and the reordered sequence came out renumbered by $\rho$.

# 3 Evaluation

## 3.1 Concordance Correlation Coefficient (CCC)

The primary evaluation metric is the Concordance Correlation Coefficient [12], which measures agreement between predicted and ground-truth engagement trajectories:

$$\mathrm{CCC}(\hat{y}, y) = \frac{2\rho\,\sigma_{\hat{y}}\,\sigma_y}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (\mu_{\hat{y}} - \mu_y)^2},$$

where $\rho$ is the Pearson correlation between predictions $\hat{y}$ and targets $y$, $\sigma_{\hat{y}}, \sigma_y$ are their standard deviations, and $\mu_{\hat{y}}, \mu_y$ are their means. $\mathrm{CCC} \in [-1, 1]$, with 1 indicating perfect concordance. Unlike Pearson correlation alone, CCC penalises both scale and location shifts.

Following the challenge protocol [13], the headline score is the **unweighted mean CCC across the four held-out test sets** - NoXi, NoXi+J, NoXi (Additional Languages), and MPIIGroupInteraction:

$$\mathrm{CCC}_{\mathrm{combined}} = \frac{1}{4} \sum_{d \in \mathcal{D}} \mathrm{CCC}_d, \qquad \mathcal{D} = \{\text{NoXi},\ \text{NoXi+J},\ \text{NoXi-Add},\ \text{MPIIGI}\}.$$

Because every test set enters with the same weight $\frac{1}{4}$, each counts equally, however many sessions or frames it contains - unlike a score pooled over all frames, in which the largest test set would dominate and a poor result on a small one would barely register. Within the mean, the scores of different test sets do offset each other one-to-one: 0.8 on NoXi and 0.2 on NoXi+J (with the other two scores fixed) give the same combined score as 0.5 on both. MultiMediate'26 additionally reports a global score that adds PInSoRo, a classification task with categorical ground truth, with weight $\frac{1}{3}$ and gives each of the four sets above $\frac{1}{6}$.

**The correlation ceiling**

CCC can be decomposed into two factors. The first measures whether the prediction moves together with the target: whether frames that are more engaged than others also receive higher predictions than those others, so that the predicted curve rises and falls where the true one does. When this holds we say that the prediction *orders* the target well. (Strictly, the Pearson correlation measures linear agreement rather than rank order; we use "ordering" loosely for it.) The second factor measures whether the prediction's level and spread *match* those of the target. Define the scale ratio and the normalised mean offset

$$\varphi := \frac{\sigma_{\hat{y}}}{\sigma_y} > 0, \qquad \omega := \frac{\mu_{\hat{y}} - \mu_y}{\sqrt{\sigma_{\hat{y}}\sigma_y}},$$

and divide numerator and denominator of the CCC by $\sigma_{\hat{y}}\sigma_y$:

$$\frac{2\rho}{\frac{\sigma_{\hat{y}}}{\sigma_y} + \frac{\sigma_y}{\sigma_{\hat{y}}} + \frac{(\mu_{\hat{y}} - \mu_y)^2}{\sigma_{\hat{y}}\sigma_y}} = \rho \cdot C_b, \qquad C_b := \frac{2}{\varphi + \varphi^{-1} + \omega^2}.$$

Here $\rho$ is the Pearson correlation, which, following [12], we term the *precision* of the predictor, and $C_b$ the bias-correction factor, its *accuracy*. The names come from measurement. An instrument is precise when its readings are consistent with each other; correspondingly, $\rho$ is high when the predictions follow the ups and downs of the target closely, even if all of them are shifted or stretched by the same amount. An instrument is accurate when its readings are centred on the true value; correspondingly, $C_b$ equals 1 when the predictions have the target's mean and spread, however well or badly they follow its ups and downs.

**Proposition.** $\mathrm{CCC}(\hat{y}, y) \le \rho$, with equality iff $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$.

*Proof.* For any $\varphi > 0$ the arithmetic-geometric mean inequality gives $\varphi + \varphi^{-1} \ge 2\sqrt{\varphi \cdot \varphi^{-1}} = 2$, with equality if and only if $\varphi = \varphi^{-1}$ ($\varphi = 1$). Since also $\omega^2 \ge 0$ with equality if and only if $\mu_{\hat{y}} = \mu_y$, the denominator of $C_b$ satisfies $\varphi + \varphi^{-1} + \omega^2 \ge 2$, hence $C_b \le 1$ and $\mathrm{CCC} = \rho\, C_b \le \rho$. Both equality conditions hold simultaneously precisely when $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$. □

**Implication.** No shifting or stretching of the predictions after training can push the CCC above $\rho$. Matching the predictions' mean and standard deviation to those of the target brings $C_b$ to 1, but it leaves $\rho$ unchanged, because $\rho$ only depends on whether the predictions go up and down together with the target. Once a model's predictions have roughly the right level and spread, the only way to improve further is therefore to raise $\rho$: to get better at telling more engaged moments from less engaged ones. This is why the architecture gives the model a longer memory (Section 1.3.1) and lets the modalities actually exchange information (Section 1.3.2), and why the training objective adds a CCC term to the frame-by-frame likelihood (Section 1.8).

## 3.2 Conditional Demographic Disparity (CDD)

To quantify fairness across demographic groups we adopt the Conditional Demographic Disparity [14, 13], which asks whether, *at the same ground-truth engagement level*, the model systematically over- or under-predicts for one group relative to another. Conditioning on $y$ is what distinguishes CDD from a raw demographic-parity gap: it separates genuine model bias from legitimate differences in the underlying engagement distributions of the groups.

For gender,

$$\mathrm{CDD}_G = \mathbb{E}\big[\hat{y} \,\big|\, \text{male},\, y\big] - \mathbb{E}\big[\hat{y} \,\big|\, \text{female},\, y\big],$$

where positive values indicate higher predictions for males than for females at the same $y$, and negative values the reverse. For language, each language $L$ is compared against the pooled

expectation across all languages at the same $y$:

$$\mathrm{CDD}_L = \mathbb{E}\big[\hat{y} \,\big|\, L,\, y\big] - \mathbb{E}\big[\hat{y} \,\big|\, y\big].$$

Both quantities are **signed** and centred at 0; values closer to zero indicate smaller disparity, and the sign indicates the direction of the bias. This is a substantive difference from an absolute-value formulation: the sign carries the information about *which* group is favoured, and averaging absolute values across groups would discard it.

**Estimation.** Since $y$ is continuous, the conditional expectations are approximated by binning the ground truth into $n_b$ equal-frequency (quantile) bins and averaging the within-bin group differences, weighted by bin prevalence:

$$\widehat{\mathrm{CDD}} = \frac{\sum_{b=1}^{n_b} |\mathcal{B}_b| \cdot \left(\bar{\hat{y}}_b^{A} - \bar{\hat{y}}_b^{B}\right)}{\sum_{b=1}^{n_b} |\mathcal{B}_b|},$$

where $\mathcal{B}_b$ is the set of frames whose target falls in bin $b$, and $\bar{\hat{y}}_b^{A}, \bar{\hat{y}}_b^{B}$ are the mean predictions of the two groups within that bin. Bins in which either group is unrepresented are skipped, since no comparison is defined there. We use $n_b = 10$.

**Applicability.** $\mathrm{CDD}_G$ requires exactly two gender codes to be present in the evaluated split, and $\mathrm{CDD}_L$ requires at least two languages; where a split does not satisfy these conditions the corresponding metric is undefined and is reported as such rather than as zero. Note also that the sign of $\mathrm{CDD}_G$ depends on which annotation code denotes male, which must be resolved against the corpus metadata; the magnitude is invariant to this choice. We acknowledge, following [13], that gender is not binary; the corpora provide only male/female self-identifications.

Because every step in the estimator is differentiable, CDD is not only measured but trained against: a squared form of it appears as the fairness term of Section 1.8. At its current weight that term has no measurable effect on $\mathrm{CDD}_G$ (Section 4).

# 4 Experimental Findings

## 4.1 Protocol

All numbers in this section are CCC values on the validation splits of NoXi (10 sessions, 577,176 frames over both roles) and NoXi+J (10 sessions, 400,896 frames); test annotations are not available to us. MPIIGroupInteraction, the fourth test set of the combined score (Section 3.1), is not addressed in this work. Unless stated otherwise, models are trained with the configuration of Section 5, separately per corpus.

**Scoring.** The network predicts overlapping windows ($L = 250$ frames, window hop 125). For evaluation, the window predictions of a session are reassembled into one sequence per role by averaging every frame over the windows that cover it (overlap-add). CCC is then computed exactly as in the official MultiMediate'26 scoring script: for each dataset, all sessions and both roles are concatenated into a single sequence, each prediction is truncated to the length of its annotation, non-finite frames are dropped, and one CCC is taken over the result. Applied to the same prediction files, our evaluation code and the official script agree to four decimals.

**Window-level scores are not session-level scores.** The validation score logged during training pools the raw overlapping windows instead of reassembled sessions, so interior frames enter twice. The two quantities differ, and not in a fixed direction: for the same checkpoints we measured 0.7942 (window-level) against 0.6987 (session-level) on NoXi, and 0.4887 against 0.5298 on NoXi+J. Checkpoints are still selected on the window-level score; re-scoring every saved checkpoint of the NoXi run at session level (epochs 10, 16, 20, 30: 0.6732, 0.6987, 0.6267, 0.6472) confirmed that the selected epoch 16 was also the best of those available.

### 4.2 Seed Variability

Training is stochastic: the initial weights, the order of training sessions and the Gumbel noise all depend on a random seed. Every configuration below was trained with at least three seeds (95, 1, 2; the two configurations that decide the final input set also 3, 4, 5), and we report the mean and the sample standard deviation. The experiment of this subsection was made with the original single-session batches of Section 4.3.

| Corpus | Window | Seed 95 | Seed 1 | Seed 2 | Mean ± sd |
|---|---|---|---|---|---|
| NoXi | 10 s | 0.6987 | **0.7111** | 0.7170 | $0.709 \pm 0.009$ |
| NoXi | 20 s | **0.7252** | 0.6947 | **0.7228** | $\mathbf{0.714 \pm 0.017}$ |
| NoXi+J | 10 s | **0.5298** | 0.5481 | 0.5082 | $0.529 \pm 0.020$ |
| NoXi+J | 20 s | 0.5005 | **0.6044** | **0.5759** | $\mathbf{0.560 \pm 0.054}$ |

Bold marks the better window length within each corpus and column. The seed-to-seed standard deviation lies between 0.01 and 0.05, so single-run differences of that size carry no information. The 20 s windows ($L = 500$, window hop 250) were tested because after the S4D-Real initialisation the SSM can retain information for up to $\sim 10^3$ steps, while a 10 s window resets the state every 250 steps. On NoXi the longer window changes nothing; on NoXi+J it raises the mean by 0.03, but with a standard deviation of 0.054 the difference is not established.

### 4.3 Batch Composition

The windows of a training batch were originally taken in order from the data stream, which walks through one session before starting the next. With a few hundred windows per session, practically every batch of 8 windows therefore came from a single session, and a single session contains exactly two people. This matters for three parts of the objective. The ordering term can then only learn the order of frames *within* one conversation, whereas the evaluation pools all sessions and also rewards placing whole sessions correctly relative to one another (Section 1.8). The fairness term is active only when that one pair happens to be of mixed gender. And consecutive batches are strongly correlated, which makes the gradient noisy.

We therefore load the active streams of all training sessions into memory once and draw every batch from a random permutation of all windows of all sessions, re-shuffled each epoch; a batch now almost always mixes several sessions. The full objective was retrained under both schemes, three seeds each. In the table, $\Delta$ is the mixed-session mean minus the single-session mean, and the number in parentheses is the $p$-value of a two-sided Welch $t$-test [22] comparing the two groups of three seed scores. Welch's test asks whether two means differ by more than the spread within each group would explain, without assuming that both groups vary equally; $p$ is the probability of seeing a difference at least this large if the batch scheme in fact made no difference, and we call a difference significant when $p < 0.05$. For $|\mathrm{CDD}_G|$ lower is better; bold marks the better scheme in each column.

| | NoXi | | NoXi+J | |
|---|---|---|---|---|
| **Training batches** | **CCC** | **mean** $\lvert\mathrm{CDD}_G\rvert$ | **CCC** | **mean** $\lvert\mathrm{CDD}_G\rvert$ |
| One session per batch | $0.709 \pm 0.009$ | 0.0069 | $0.529 \pm 0.020$ | 0.0249 |
| Mixed sessions | $\mathbf{0.724 \pm 0.008}$ | **0.0064** | $\mathbf{0.599 \pm 0.029}$ | **0.0121** |
| $\Delta$ ($p$, Welch) | +0.015 (0.10) | | +0.070 (0.03) | |

Mixing sessions improves NoXi+J significantly and NoXi in the same direction. The gender disparity on NoXi+J halves (per seed 0.009, 0.016, 0.012, against 0.000, 0.039, 0.036 before), but with three seeds the difference is not significant ($p = 0.41$). One NoXi+J seed was by mistake trained twice concurrently into the same directory; the two copies scored 0.585 and 0.624, of

which the table uses the later one, and the improvement remains significant with either. All results from here on use mixed batches unless stated otherwise.

### 4.4 Ablation of the Training Objective

An *ablation* removes one component of a system at a time and measures how much the result changes; a component whose removal lowers the score was contributing to it. Here the components are those of the training objective of Section 1.8, which has four terms: the per-frame Beta likelihood, the ordering (CCC) term, per-modality supervision and the fairness penalty. We trained one *variant* per component, identical to the full model except that this component is switched off by setting its weight to zero (10 s windows, core features, three seeds per variant). Besides the three auxiliary terms, we also switched off the reweighting of the likelihood ($\beta_w = 0$). The reweighting is not a separate term but the per-frame weight $w_t = \kappa_t^{-\beta_w}$ inside both likelihood terms; with $\beta_w = 0$ every frame gets weight 1 and the plain likelihood remains. The per-frame likelihood itself is the one term we never remove, because it is the only term that fits the whole predicted distribution of the multimodal heads - its width as well as its mean. The ordering and fairness terms look only at the predicted means, so without the likelihood nothing would determine the concentration $\kappa = \alpha + \beta$, i.e. how wide the predicted distribution is, and the uncertainty used by test-time adaptation would be meaningless.

We ran the whole ablation twice, once under each batch scheme of Section 4.3, since the scheme turned out to matter. Each entry is the difference between the variant's three-seed mean and that of the full objective under the same scheme, with the Welch $p$-value of Section 4.3 in parentheses. A negative difference means that the score dropped when the component was removed, i.e. that the component was helping. Bold marks the largest drop in each column, and * marks $p < 0.05$.

| | **One session per batch** | | **Mixed sessions** | |
|---|---|---|---|---|
| **Variant** | **NoXi** | **NoXi+J** | **NoXi** | **NoXi+J** |
| Full objective (mean ± sd) | $0.709 \pm 0.009$ | $0.529 \pm 0.020$ | $0.724 \pm 0.008$ | $0.599 \pm 0.029$ |
| No ordering ($\lambda_c = 0$) | $-0.020$ (0.20) | $\mathbf{-0.132}$ (0.048)* | $+0.008$ (0.41) | $-0.011$ (0.69) |
| No per-modality ($\lambda_u = 0$) | $-0.035$ (0.26) | $-0.031$ (0.44) | $-0.008$ (0.39) | $-0.009$ (0.75) |
| No reweighting ($\beta_w = 0$) | $\mathbf{-0.037}$ (0.22) | $-0.034$ (0.13) | $+0.002$ (0.87) | $+0.004$ (0.85) |
| No fairness ($\lambda_f = 0$) | $+0.001$ (0.91) | $-0.019$ (0.69) | $\mathbf{-0.028}$ (0.06) | $\mathbf{-0.026}$ (0.26) |

**Ordering term.** With single-session batches, removing $\lambda_c(1 - \widehat{\text{CCC}})$ causes the largest loss and the only individually significant one: on NoXi+J the mean falls from 0.529 to 0.396. With mixed batches its removal has no measurable effect. A plausible explanation, which we have not tested directly, is that a single-session batch shows the likelihood only one conversation at a time, so it cannot learn that one session is on the whole more engaged than another, and the ordering term compensates for this; once a batch mixes sessions, the likelihood sees those differences itself. Whatever the mechanism, the result shows that the value of a loss term cannot be judged independently of how the batches it is computed on are composed.

**Per-modality supervision and likelihood reweighting.** With single-session batches, removing the per-modality term or switching off the reweighting lowers the mean by 0.03 to 0.04 on both corpora - not significant individually, but consistent in all four cases. With mixed batches both effects shrink below 0.01. Neither term is shown to help in the final setting; neither is shown to hurt.

**Fairness term.** With single-session batches, removing it changes nothing measurable. With mixed batches, removing it lowers the mean by about 0.027 on both corpora ($p = 0.06$ and 0.26); in other words, keeping the term slightly *raised* accuracy, although it was added for

fairness and not for accuracy. A possible explanation is that it acts as a regulariser - an extra constraint that keeps the model from fitting peculiarities of the training data too closely and so helps it generalise - but we have not tested this, and the effect is not significant. Its effect on the quantity it was designed for is not measurable in either scheme (lower is better; bold marks the lowest value in each column):

| **Batches** | **Variant** | **NoXi** mean $\lvert\mathrm{CDD}_G\rvert$ | **NoXi+J** mean $\lvert\mathrm{CDD}_G\rvert$ |
|---|---|---|---|
| One session | $\lambda_f = 0.05$ | 0.0069 | 0.0249 |
| One session | $\lambda_f = 0$ | 0.0118 | 0.0258 |
| Mixed | $\lambda_f = 0.05$ | **0.0064** | **0.0121** |
| Mixed | $\lambda_f = 0$ | 0.0074 ($p = 0.89$) | 0.0163 ($p = 0.53$) |

**What the ablation can and cannot show.** With three seeds per group and a seed standard deviation of 0.01 to 0.03, only differences of roughly 0.04 or more are detectable. Moreover, when eight comparisons are made at once, the chance that at least one of them reaches $p < 0.05$ by luck alone is far larger than 5%. The Holm correction [23] compensates for this: the $p$-values are sorted, the smallest is multiplied by the number of comparisons (8), the next by 7, and so on, and a result counts as significant only if its corrected value stays below 0.05. In the mixed-batch ablation the smallest $p$-value, 0.055 (shown as 0.06 in the table), becomes 0.44, so none of the eight differences is significant. We therefore keep the full objective: no term is shown to hurt, and dropping one on the basis of these differences would itself be a decision made on noise.

### 4.5 Input Features

The model so far reads the four core streams (eGeMAPS, W2v-BERT, OpenFace2, OpenPose). All ten feature types of the challenge release are available for every split of both corpora, test included. We added them in groups to the core set: text (the XLM-RoBERTa embedding of the transcript), image embeddings (DINO, ImageBind, Swin), video embeddings (VideoMAE, CLIP), and all ten together. The core set and core plus text were trained with six seeds, the others with three; $\Delta$ and $p$ (Welch, as in Section 4.3) are relative to the core set. Bold marks the best value in each column.

| | **NoXi** | | | **NoXi+J** | | |
|---|---|---|---|---|---|---|
| **Inputs** | **Mean ± sd** | $\Delta$ | $p$ | **Mean ± sd** | $\Delta$ | $p$ |
| Core 4 | $0.726 \pm 0.010$ | – | – | $0.593 \pm 0.023$ | – | – |
| + text | $\mathbf{0.731 \pm 0.020}$ | $\mathbf{+0.005}$ | 0.58 | $\mathbf{0.641 \pm 0.018}^*$ | $\mathbf{+0.047}^*$ | **0.003** |
| + image | $0.701 \pm 0.017$ | $-0.025$ | 0.11 | $0.593 \pm 0.020$ | $-0.001$ | 0.97 |
| + video | $0.712 \pm 0.027$ | $-0.014$ | 0.47 | $0.509 \pm 0.033$ | $-0.084$ | 0.03 |
| All ten | $0.714 \pm 0.017$ | $-0.012$ | 0.34 | $0.641 \pm 0.038$ | $+0.048$ | 0.15 |

* All ten streams score marginally higher on NoXi+J (0.6412 against 0.6406), far below the seed-to-seed spread, while taking about 2.5 times longer to train, varying twice as much across seeds and scoring lower on NoXi; we therefore count core plus text as the best input set.

The text stream is the only addition that helps, and on NoXi+J it does so significantly. The two input sets that contain it are the two best on NoXi+J, and all three sets without it are at or below the core set; adding all ten streams brings nothing over text alone and takes about 2.5 times longer to train. On NoXi the text stream is neutral. The image embeddings add nothing, and the video embeddings significantly harm NoXi+J. We adopt core plus text as the final input set.

The window-length comparison of the seed-variability experiment was made with single-session batches and core features and has not been repeated in the final setting; the final configuration keeps 10 s windows.

### 4.6 Learned versus Fixed Modality Ordering

The ordering layer of Section 2.2 is a contribution only if it does better than a fixed order. We trained the final configuration (core features plus text, mixed batches, full objective) with the meta-network removed and $P$ replaced by a constant permutation matrix, for four fixed orders of the $M = 10$ streams, with three seeds each (95, 1, 2), and compared each with the six seeds of the learned ordering by Welch's $t$-test, as in Section 4.5. The four orders are: (i) role-blocked, the five streams of the expert followed by the five of the novice, feature types in alphabetical order (the order in which the code stacks the streams before $P$ is applied); (ii) role-interleaved, the expert and novice streams of each feature type next to each other; (iii) and (iv) two random orders, drawn once and kept fixed for all seeds. An order and its reverse are not both included: the forward and backward scans of the bidirectional block have the same structure and separately initialised parameters, so reversing the sequence only exchanges their roles.

| | NoXi | | | NoXi+J | | |
|---|---|---|---|---|---|---|
| **Ordering** | **Mean ± sd** | $\Delta$ | $p$ | **Mean ± sd** | $\Delta$ | $p$ |
| Learned (6 seeds) | $\mathbf{0.731 \pm 0.020}$ | – | – | $0.641 \pm 0.018$ | – | – |
| Role-blocked | $0.702 \pm 0.023$ | $-0.028$ | 0.15 | $0.672 \pm 0.006$ | $+0.031$ | $0.007^*$ |
| Role-interleaved | $0.717 \pm 0.006$ | $-0.014$ | 0.17 | $\mathbf{0.676 \pm 0.027}$ | $+0.036$ | 0.13 |
| Random 1 | $0.710 \pm 0.008$ | $-0.021$ | 0.06 | $0.647 \pm 0.014$ | $+0.006$ | 0.59 |
| Random 2 | $0.718 \pm 0.016$ | $-0.013$ | 0.34 | $0.667 \pm 0.007$ | $+0.027$ | $0.016^*$ |
| All fixed (12 runs) | $0.712 \pm 0.014$ | $-0.019$ | 0.07 | $0.666 \pm 0.018$ | $+0.025$ | $0.02^*$ |

$\Delta$ and $p$ (Welch) are relative to the learned ordering. Bold marks the best mean in each column; $^*$ marks $p < 0.05$ before correction.

The learned ordering does not do better than a fixed order. On NoXi every fixed order scores somewhat lower ($-0.013$ to $-0.028$), but none of the differences is significant, and pooled over the four orders the gap is $-0.019$ ($p = 0.07$). On NoXi+J the picture is reversed: every fixed order scores *higher*, by up to 0.036, and pooled over the four orders by 0.025 ($p = 0.02$). Two single comparisons reach $p < 0.05$, but after the Holm correction over the eight comparisons the smallest corrected value is 0.057, so no single fixed order is shown to differ from the learned one. The four fixed orders differ from each other by about as much as seeds do, so on these data the choice of order matters little, and learning it brings no measurable gain; on NoXi+J it costs accuracy. The fixed models also lack the meta-network and the Gumbel noise, so we cannot separate the effect of the ordering from that of removing the noise.

**What the learned $P$ looks like.** We read out $P$ on the validation windows for one learned model per corpus (seed 95). It is far from a permutation: the largest entry of a row averages 0.73 on both corpora, only 55% (NoXi) and 49% (NoXi+J) of rows have an entry above 0.9, and rounding each row to its largest entry gives a valid permutation for only 0.4% and 2.1% of the windows (1,006 and 1,431 distinct rounded orders). Moreover, the rows of the averaged $P$ do not sum to one but to between 0.6 and 2.0 on NoXi and between 0.86 and 1.37 on NoXi+J. With $n_{\text{iter}} = 10$, the final Sinkhorn pass normalises the columns, but the iteration has not converged, so the matrix is column-stochastic without being doubly stochastic: some positions receive the weight of about two streams, others of about half a stream. In practice the layer therefore acts as a learned, input-dependent soft mixing of streams into positions rather than as an ordering. Whether the unscaled scores of Section 2.2 are the reason for the slow convergence we have not tested.

**Four problems in the first implementation.** Inspecting the layer against its intended mathematics revealed four faults, two of which penalise only the learned ordering and therefore bias the comparison above against it.

1. **The heads saw a changing layout.** The fused slots were concatenated in scan order and read by one dense layer per role. Under an input-dependent $P$, which stream occupied which part of the heads' input changed from window to window, so the read-out weights could not specialise; with a fixed order the layout never changes. Corrected by mapping the slots back to stream order with $P^\top$ before the heads (Section 1.3.2).
2. **Inference ran at a different temperature.** The best checkpoint was selected at the temperature of its epoch, but inference always used $\tau_\infty = 0.1$, giving a sharper $P$ than the model was trained or selected with. Fixed orders do not depend on $\tau$. Corrected by storing the temperature of the selected epoch with the checkpoint and using it at inference (Section 2.2).
3. **Sinkhorn had not converged.** With 10 iterations and unscaled scores, $P$ was column-stochastic but not doubly stochastic. Corrected by the $1/\sqrt{d_s}$ scaling and 50 iterations.
4. **The score matrix had no notion of position.** With data-dependent queries, $Z_{ij}$ compared stream $i$ with stream $j$ rather than scoring stream $j$ for position $i$, and the result depended on how the streams were numbered. Corrected by one learned query per position.

All other results in this document were obtained with the first implementation.

**Effect of the inference temperature alone.** Re-scoring the six existing learned models per corpus at the temperature of their selected epoch (0.29 to 0.49 on NoXi, 0.36 to 0.95 on NoXi+J) instead of $\tau_\infty = 0.1$, without retraining, changes the validation CCC by $-0.001$ on NoXi ($0.729 \pm 0.022$; paired $t$-test over the same models, $p = 0.15$) and by $+0.005$ on NoXi+J ($0.645 \pm 0.017$; $p = 0.03$, all six seeds improve). The effect is real on NoXi+J but an order of magnitude smaller than the gap to the fixed orders.

**Corrected layer.** With all four corrections (position queries, scaled scores, $n_{\text{iter}} = 50$, unsorting before the heads, inference at the selected temperature), trained with the same configuration and seeds (95, 1, 2):

| | NoXi | | | NoXi+J | | |
|---|---|---|---|---|---|---|
| **Compared with** | **Other** | $\Delta$ | $p$ | **Other** | $\Delta$ | $p$ |
| Role-blocked | 0.702 | +0.014 | 0.44 | 0.672 | −0.030 | 0.21 |
| Role-interleaved | 0.717 | −0.000 | 0.98 | 0.676 | −0.034 | 0.21 |
| Random 1 | 0.710 | +0.007 | 0.57 | 0.647 | −0.004 | 0.83 |
| Random 2 | 0.718 | −0.001 | 0.94 | 0.667 | −0.025 | 0.27 |
| All fixed (12 runs) | 0.712 | +0.005 | 0.68 | 0.666 | −0.023 | 0.29 |
| First learned layer (6 seeds) | 0.729 | −0.013 | 0.39 | 0.645 | −0.003 | 0.88 |

The corrected layer scores $0.717 \pm 0.017$ on NoXi and $0.642 \pm 0.029$ on NoXi+J. $\Delta$ is the corrected layer minus the other configuration; $p$ from Welch's test. The first learned layer is scored at the selected temperature.

The corrections do not change the conclusion. The corrected layer is level with the fixed orders on NoXi and below them on NoXi+J, and it does not differ from the first learned layer on either corpus; no comparison approaches significance (smallest Holm-corrected $p = 1.0$ over the eight comparisons with fixed orders).

The read-out of $P$ (seed 95) shows that the corrections changed the layer itself. On NoXi, $P$ is now close to a permutation: the mean largest row entry is 0.89, 69% of rows have an entry above 0.9, and rounding gives a valid permutation for 85% of the windows (previously 0.4%), with 754 distinct orders. The ordering is still strongly input-dependent; the most frequent order covers only 2% of the windows. Sinkhorn has still not fully converged on NoXi (row sums 0.50 to 1.33): the learned position queries grew during training until the per-row spread of $Z/\tau$ reached

a median of 76, at which point 50 iterations are not enough. On NoXi+J the layer stayed soft (mean largest row entry 0.53, 10% valid rounded permutations, 1,172 distinct orders), with a median spread of $Z/\tau$ of 8.3 and row sums between 0.92 and 1.05.

**Why ordering cannot show a gain here.** The four fixed orders, which include very different arrangements of roles and feature types, differ from each other by at most 0.016 on NoXi and 0.030 on NoXi+J, which is about the seed standard deviation. A layer that always picked the best of them could therefore gain at most this much over a fixed order, below the difference of about 0.04 that three seeds can detect (Section 4.5). On these data the order in which the inter-modal block scans the streams matters little, and a learned ordering has little to improve.

**When the scan cannot depend on the order.** Consider a simplified bidirectional scan with no state decay ($\bar{A} = 1$), no convolution, and the same $\bar{B}_j$, $C_j$ for the forward and the backward direction, each computed from stream $j$ alone. The forward state at position $i$ is $h_i^{\rightarrow} = \sum_{j \le i} \bar{B}_j x_j$, the backward state $h_i^{\leftarrow} = \sum_{j \ge i} \bar{B}_j x_j$, so

$$y_i = C_i^{\top}\left(h_i^{\rightarrow} + h_i^{\leftarrow}\right) = C_i^{\top}\Big(\sum_{j=1}^{M} \bar{B}_j x_j + \bar{B}_i x_i\Big),$$

which depends only on stream $i$ itself and on the sum over all streams. Renumbering the positions renumbers the outputs in the same way and changes nothing else: the scan is permutation-equivariant, and after unsorting, its output does not depend on the order at all. Our block departs from this case in three places: $\bar{A} = \exp(\Delta A) < 1$, a depthwise convolution of width 4 over neighbouring positions, and separate forward and backward parameters. Order can only matter through these, and how much it does is an empirical question, which the following tests answer.

**Planned: testing that the order does not matter.** A non-significant difference is not evidence that there is none, so the claim is tested as an equivalence, with the margin fixed in advance at $\delta = 0.02$ CCC (about one seed standard deviation; smaller differences are of no practical consequence). Equivalence within $\pm\delta$ is shown by two one-sided tests (TOST) at $\alpha = 0.05$, i.e. when the 90% confidence interval of the difference lies inside $(-\delta, \delta)$; for every comparison we also report the smallest margin the data establish.

- *Training-time order.* Three more seeds (3, 4, 5) for each fixed order on both corpora, six per order in all. One-way ANOVA with variance components (seed and order standard deviation, with an upper confidence bound on the latter), and for each order a Welch TOST against the runs of the other three orders.
- *Inference-time order.* Every trained fixed-order model is run, without retraining, with the scan in six other orders (the three other fixed orders and three further random orders); the outputs are mapped back to the layout the heads were trained on, so only the scan order changes. The difference $\Delta$ to the same model in its own order is paired, so seed variance drops out. The unit of analysis is the trained model ($\Delta$ averaged over its probes, 12 models per corpus), tested with a one-sample TOST.
- *Mechanism.* The distribution of $\bar{A}$ in the trained inter-modal block on the validation windows, and the share $\bar{A}^{M-1}$ of the state that survives the whole sequence of streams, in each direction.

### 4.7 Cross-Corpus Transfer: a Shift in Label Scale

The analyses of this and the next subsection were made with models trained on single-session batches and core features (Sections 4.3 and 4.5); we have not repeated them in the final setting, but nothing in them depends on those choices. A model trained on one corpus and evaluated

on the other scores close to zero. The decomposition CCC $= \rho\, C_b$ of Section 3.1 shows where the loss arises:

| **Trained on** | **Evaluated on** | **CCC** | $\rho$ | $C_b$ | **mean** $\hat{y}$ | **mean** $y$ |
|---|---|---|---|---|---|---|
| NoXi | NoXi | **0.699** | **0.703** | 0.995 | 0.584 | 0.570 |
| NoXi | NoXi+J | 0.092 | 0.422 | 0.219 | 0.577 | 0.834 |
| NoXi+J | NoXi+J | 0.530 | 0.530 | **0.999** | 0.838 | 0.834 |
| NoXi+J$^\dagger$ | NoXi | 0.143 | 0.445 | 0.321 | 0.815 | 0.570 |

$^\dagger$ An earlier NoXi+J model, trained with learning rate $5 \times 10^{-4}$.

Across corpora the correlation stays at 0.42 to 0.45, against 0.53 to 0.70 within a corpus: the models still order frames reasonably well on unseen data. What collapses is the accuracy factor $C_b$, and the reason is visible in the means. The NoXi+J annotations average 0.834, the NoXi annotations 0.570, and each model continues to predict at the level of the corpus it was trained on. Take the NoXi model on NoXi+J: its predictions rise and fall roughly where the annotations do, but the whole predicted curve sits about 0.26 below the annotated one (mean 0.577 against 0.834). Measured against the spread of the two curves (standard deviations 0.090 and 0.102), this offset is large, $\omega \approx -2.7$, and since $C_b = \dfrac{2}{\varphi + \varphi^{-1} + \omega^2}$ with $\varphi + \varphi^{-1} \approx 2.02$, it alone pulls $C_b$ down to about $2/9.2 \approx 0.22$; the difference in spread would by itself only cost about 1%. The NoXi+J model on NoXi shows the mirror image, predicting 0.245 too high. The two corpora therefore differ mainly in the *scale* of their engagement labels - a shift in the label distribution, not in the input features. Our data cannot tell whether this reflects genuinely more engaged participants or a different use of the rating scale by the annotators of the two corpora.

Two consequences follow. First, by the proposition of Section 3.1, an affine recalibration of the predictions to the target's mean and standard deviation would raise the cross-corpus CCC to $\rho$, that is to about 0.42 and 0.45, but no further; it would require knowing the target corpus's label statistics, which a deployed model does not. In practice, corpus-specific models are therefore required, which is also the protocol of the official baseline. Second, the shift is invisible to the test-time adaptation of Section 1.6: its objective compares the model's heads with each other on unlabelled data and contains no information about the target corpus's label scale.

### 4.8 Test-Time Adaptation

Test-time adaptation was evaluated with the implementation of Section 1.6, in the cross-corpus setting it was designed for (NoXi model on NoXi+J) and in-corpus (NoXi+J model on NoXi+J).

| **Setting** $(q_m, q_u, \eta_{\mathrm{TTA}})$ | **NoXi → NoXi+J selected** | **NoXi → NoXi+J CCC** | **NoXi+J → NoXi+J selected** | **NoXi+J → NoXi+J CCC** |
|---|---|---|---|---|
| No adaptation | – | 0.0922 | – | 0.5298 |
| $(0.3,\ 0.7,\ 10^{-4})$ | $\approx 2\%^\ddagger$ | 0.0921 | – | **0.5339** |
| $(0.5,\ 0.5,\ 10^{-4})$ | 16.7% | 0.0927 | 17.2% | 0.5332 |
| $(0.5,\ 0.5,\ 10^{-3})$ | 14.7% | **0.0953** | 18.4% | 0.4178 |
| $(0.3,\ 0.7,\ 10^{-3})$ | 4.5% | 0.0897 | 5.5% | 0.4796 |

$^\ddagger$ Estimated from the first three sessions; the fraction was not logged for the full run. Bold marks the best CCC in each column.

The row "No adaptation" is the trained model on its own. Within its corpus it generalises to the unseen validation sessions as expected (0.53 on NoXi+J); across corpora it fails (0.09), for the label-scale reason of the previous subsection. Adaptation does not change this picture. No setting changes the cross-corpus score by more than 0.004. In-corpus, the smaller learning rate

is neutral within seed noise (+0.004) and the larger one is harmful (−0.05 to −0.11). At the design quantiles (0.3, 0.7) the filter selects only 2 to 6% of the windows, far below the 9% that two independent conditions of 30% each would give; presumably the fused and the unimodal uncertainties tend to rise and fall together, which makes the joint condition rare, although we have not measured this correlation. Taken together with the previous subsection, the result is a clear negative one: in its current form the adaptation neither closes the cross-corpus gap, which lies in the label scale, nor improves in-corpus predictions, and with a larger learning rate it makes them worse. All other results in this document are obtained without it.

**Next: adaptation on a single person.** In the runs above, the stream being adapted to is a whole session, and the sessions of a split come from many participants. The shift the adaptation meets is therefore a mixture of differences between persons and between corpora, and the latter is dominated by the label scale, which the adaptation, having no labels, cannot observe. We will next run it on the data of one participant at a time: only that person's multimodal and unimodal uncertainties enter the filter, only that person's heads provide the loss, the parameters are reset when the participant changes, and the effect is measured on that participant's frames against the unadapted model. If adaptation does not help there either, the uncertainty estimates of methods 2 and 3 (Section 1.7) will replace the parametric variance in the filter.

### 4.9 Ensembling

For each corpus we average, frame by frame, the predictions of the six seeds of the final configuration (core features plus text, mixed batches, full objective, 10 s windows). Every member is trained on the training split only, and all six are used; no member is chosen by its validation score, since selecting the best single run would mostly select the luckiest seed.

**Label-free rescaling.** Averaging cancels the members' independent errors, but it also flattens the prediction. Where the members disagree about how high or low a frame should be, their average lands in between, so the averaged curve rises and falls at the same places as the members' curves, but not as far. On NoXi+J the members' predictions have a standard deviation of 0.086 on average, the ensemble's only 0.078, against 0.102 for the annotations, and this too-small spread lowers $C_b$. We therefore stretch the ensemble's curve vertically around its own mean until its spread equals the members' typical spread. The stretch does not change which frames are predicted higher than which, only how far apart they are:

$$\hat{y}'_t = \mu_{\text{ens}} + \left(\hat{y}_t - \mu_{\text{ens}}\right) \frac{\bar{\sigma}_{\text{mem}}}{\sigma_{\text{ens}}}.$$

Here $\hat{y}_t$ is the ensemble prediction (the frame-wise average of the members), and $\mu_{\text{ens}}$ and $\sigma_{\text{ens}}$ are the mean and standard deviation of this averaged curve over all frames of the split. $\bar{\sigma}_{\text{mem}}$ is computed the other way round: first the standard deviation of each member's own curve, then the average of these six numbers. The two orders of operation give different results for the spread - the spread of an average is smaller than the average spread, unless all members agree perfectly - and that difference is exactly what the stretch undoes; on NoXi+J the factor is $0.086/0.078 \approx 1.10$. For the level, both orders give the same number, since the mean of the averaged curve equals the average of the members' means, so the mean is left where it is. The result is clipped to $[0, 1]$. Only predictions enter this transformation, never annotations, so it applies unchanged to the test split. Being affine, it leaves $\rho$ untouched and can only move $C_b$.

| | NoXi | | | NoXi+J | | |
|---|---|---|---|---|---|---|
| **Predictor** | **CCC** | $\rho$ | $C_b$ | **CCC** | $\rho$ | $C_b$ |
| Single model (mean of 6 seeds) | 0.731 | – | – | 0.641 | – | – |
| Ensemble of 6 seeds | 0.756 | **0.761** | 0.993 | 0.696 | **0.744** | 0.936 |
| Ensemble of 6 seeds, rescaled | **0.758** | **0.761** | **0.996** | **0.714** | **0.744** | **0.959** |

Ensembling raises the correlation itself, which is precisely the factor that the correlation ceiling of Section 3.1 identifies as the only remaining margin; the rescaling then recovers part of the accuracy lost to averaging. On NoXi+J a gap in $C_b$ remains, almost entirely from the level: the ensemble predicts 0.812 on average against 0.834 in the annotations. Two caveats apply: the decision to rescale was made on the validation split, so the final numbers are slightly optimistic; and an ensemble is a statement about the best system we can build, not about the method. We therefore report the single-model mean over seeds as the estimate of the method and the rescaled six-seed ensemble as the submitted system.

### 4.10 Summary

| **Validation CCC** | **NoXi** | **NoXi+J** |
|---|---|---|
| Single model, single-session batches, core features (3 seeds) | 0.709 | 0.529 |
| + mixed-session batches (Section 4.3, 3 seeds) | 0.724 | 0.599 |
| + text stream (Section 4.5, 6 seeds) | 0.731 | 0.641 |
| Six-seed ensemble, rescaled (submitted system) | **0.758** | **0.714** |
| *For reference, MultiMediate'26 test leaderboard* | | |
| Official baseline | 0.55 | 0.31 |
| Best entry | 0.82 | 0.66 |

Our numbers are on the validation split and the leaderboard's on the test split, so the comparison is indicative only. The submitted test predictions (NoXi test-base and test-additional, NoXi+J test) are produced by the same rescaled six-seed ensemble.

# 5 Hyperparameters

The following values define the reference configuration. All are exposed as command-line overrides.

| Symbol | Description | |
|---|---|---|
| *Data* | | |
| $L$ | Window length (frames; 10 s at 25 Hz) | |
| – | Window hop (frames; 50% overlap) | |
| – | Feature types $\lvert\mathcal{F}\rvert$ (core 4 + text) | |
| $M$ | Role-feature pairs $= 2\lvert\mathcal{F}\rvert$ | |
| – | Training batches | mixed |
| $\varepsilon$ | Target clipping bound | |
| *Architecture* | | |
| $C'$ | Shared channel width after projection | |
| $N$ | SSM state dimension | |
| $D_C$ | Depthwise convolution kernel size (intra-modal) | |
| – | Depthwise convolution kernel size (inter-modal, scan over $M$) | |
| $d_h$ | Beta head hidden width | |
| – | Total parameters (final configuration) | ≈ |
| $\Delta_{\min}$ | Minimum timestep at initialisation | |
| $\Delta_{\max}$ | Maximum timestep at initialisation | |
| – | $A$ initialisation scheme | S |
| *Gumbel-Sinkhorn* | | |
| $d_s$ | Query/key projection dimension | |
| $n_{\text{iter}}$ | Sinkhorn normalisation iterations | 10; 50 in the corrected layer (Sect |
| $\tau_0$ | Initial temperature | |
| $\tau_\infty$ | Minimum temperature | |
| $\gamma$ | Temperature decay rate per epoch | |
| *Optimisation* | | |
| $\eta$ | Learning rate (AdamW) | 2 |
| – | Weight decay (AdamW) | |
| – | Batch size | |
| – | Maximum epochs | |
| – | Early-stopping patience (epochs, on val CCC) | |
| – | Seeds (final configuration: 6) | 95, 1, 2 |
| $\lambda_c$ | Weight of the ranking (CCC) term | |
| $\lambda_u$ | Unimodal deep-supervision weight (before $1/M$) | |
| $\lambda_f$ | Fairness penalty weight | |
| $\beta_w$ | Likelihood reweighting exponent | |
| *Test-time adaptation* | | |
| $\lambda$ | Pseudo-label supervision weight in $\mathcal{L}_{TTA}$ | |
| $q_m$ | Quantile of the multimodal-uncertainty threshold | |
| $q_u$ | Quantile of the unimodal-uncertainty threshold | |
| $B_{\text{inf}}$ | Windows per inference batch | |
| $\eta_{\text{TTA}}$ | Adaptation learning rate (Adam) | |
| *Evaluation* | | |
| $n_b$ | Ground-truth bins for CDD estimation | |

The learning rate was lowered from the initial $10^{-3}$: at that rate a full run reached its best validation CCC at epoch 2 and then stalled while the training loss kept falling, the signature of steps that are too large once input standardisation lets gradients propagate properly. All results in Section 4 use $\eta = 2 \times 10^{-4}$ and patience 15.

# References


[1] Ziyu Jia et al., *A Multimodal BiMamba Network with Test-Time Adaptation for Emotion Recognition Based on Physiological Signals*, 39th Conference on Neural Information Processing Systems (NeurIPS 2025).

[2] Albert Gu and Tri Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, arXiv preprint arXiv:2312.00752 (2024).

[3] Gonzalo Mena et al., *Learning Latent Permutations with Gumbel-Sinkhorn Networks*, International Conference on Learning Representations (ICLR 2018).

[4] Eric Jang, Shixiang Gu, and Ben Poole, *Categorical Reparameterization with Gumbel-Softmax*, International Conference on Learning Representations (ICLR 2017).

[5] Emil Julius Gumbel, *Statistical Theory of Extreme Values and Some Practical Applications*, National Bureau of Standards Applied Mathematics Series, 33 (1954).

[6] Garrett Birkhoff, *Tres observaciones sobre el algebra lineal*, Universidad Nacional de Tucumán, Revista, Serie A, 5:147–151 (1946).

[7] Aitchison, J., & Shen, S. M. (1980). *Logistic-normal distributions: Some properties and uses.* Biometrika, 67(2), 261–272.

[8] P. Kumaraswamy, *A generalized probability density function for double-bounded random processes*, Journal of Hydrology, 46, 79–88 (1980).

[9] Guilherme Pumi, Cleber Rauber, and Fabio M. Bayer, *Kumaraswamy regression model with Aranda-Ordaz link function*, TEST: An Official Journal of the Spanish Society of Statistics and Operations Research, Springer;Sociedad de Estadística e Investigación Operativa, 29(4), 1051–1071, December (2020).

[10] Yaniv Romano, Evan Patterson, and Emmanuel J. Candès, *Conformalized Quantile Regression*, Advances in Neural Information Processing Systems (NeurIPS 2019), 32 (2019).

[11] Isaac Gibbs and Emmanuel Candès, *Adaptive Conformal Inference Under Distribution Shift*, Advances in Neural Information Processing Systems (NeurIPS 2021), 34, 1660–1672 (2021).

[12] Lawrence I-Kuei Lin, *A Concordance Correlation Coefficient to Evaluate Reproducibility*, Biometrics, 45(1), 255–268 (1989).

[13] Daksitha Withanage Don, Marius Funk, Michal Balazia, Huajian Qiu, Shogo Okada, François Brémond, Jan Alexandersson, Andreas Bulling, Elisabeth André, and Philipp Müller, *MultiMediate '25: Cross-cultural Multi-domain Engagement Estimation*, Proceedings of the 33rd ACM International Conference on Multimedia (MM '25), 14150–14155 (2025). doi:10.1145/3746027.3762076.

[14] Sandra Wachter, Brent Mittelstadt, and Chris Russell, *Why fairness cannot be automated: Bridging the gap between EU non-discrimination law and AI*, Computer Law and Security Review, 41, 105567 (2021). doi:10.1016/j.clsr.2021.105567.

[15] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton, *Layer Normalization*, arXiv preprint arXiv:1607.06450 (2016).

[16] Sergey Ioffe and Christian Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, International Conference on Machine Learning (ICML 2015), 448–456.

[17] Angelo Cafaro, Johannes Wagner, Tobias Baur, Soumia Dermouche, Mercedes Torres Torres, Catherine Pelachaud, Elisabeth André, and Michel F. Valstar, *The NoXi Database: Multimodal Recordings of Mediated Novice-Expert Interactions*, Proceedings of the 19th ACM International Conference on Multimodal Interaction, 350–359 (2017). doi:10.1145/3136755.3136780.

[18] Marius Funk, Shogo Okada, and Elisabeth André, *Multilingual Dyadic Interaction Corpus NoXi+J: Toward Understanding Asian-European Non-verbal Cultural Characteristics and their Influences on Engagement*, Proceedings of the ACM International Conference on Multimodal Interaction, 224–233 (2024). doi:10.1145/3678957.3685757.

[19] Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré, *On the Parameterization and Initialization of Diagonal State Space Models*, Advances in Neural Information Processing Systems (NeurIPS 2022), 35, 35971–35983.

[20] Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab, *Signals and Systems*, 2nd edition, Prentice Hall (1997).

[21] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius, *On the Pitfalls of Heteroscedastic Uncertainty Estimation with Probabilistic Neural Networks*, International Conference on Learning Representations (ICLR 2022).

[22] B. L. Welch, *The Generalization of "Student's" Problem when Several Different Population Variances are Involved*, Biometrika, 34(1–2), 28–35 (1947).

[23] Sture Holm, *A Simple Sequentially Rejective Multiple Test Procedure*, Scandinavian Journal of Statistics, 6(2), 65–70 (1979).